

VT Sistem Gerçekleme Ders Notları- #4

Remote: Kullanıcıdan gelen JDBC isteklerini karşılar.
Planner: SQL ifadesi için işleme planı oluşturur ve karşılık gelen ilişkisel cebir ifadesini oluşturur.
Parse: SQL ifadesindeki tablo, nitelik ve ifadeleri ayrıştırır.
Query: Algebra ile ifade edilen sorguları gerçekler.
Metadata: Tablolara ait katalog bilgilerini organize eder.
Record: disk sayfalarına yazma/okumayı kayıt seviyesinde gerçekler.
Transaction&Recovery: Eşzamanlılık için gerekli olan disk sayfa erişimi kısıtlamalarını organize eder ve veri kurtarma için kayıt_defteri (<i>log</i>) dosyalarına bilgi girer.
Buffer: En sık/son erişilen disk sayfalarını ana hafıza tampon bölgede tutmak için gerekli işlemleri yapar.
Log: Kayıt_defterine bilgi yazılmasını ve taranması işlemlerini düzenler.
File: Dosya blokları ile ana hafıza sayfaları arasında bilgi transferini organize eder.

Hareket (*transaction*) yönetimi (HY)

■ Kurtarma(*recovery*):KY

■ Eşzamanlılık (*concurrency*) : EY

Konu başlıkları

- Hareketin tanımlanması
 - ACID, hareketin durumları
- HY olmazsa ne olur?
 - Autocommit(true) modu
- SimpleDB’de Örnek Hareket Uygulaması
- Eşzamanlılık ne demek?
- Eşzamanlılık yönetimi (EY) olmazsa ne olur?
 - Dirty read, lost update, phantom, nonrepeatable read, yalıtım seviyeleri
- HATA tipleri nelerdir?
- Veri kurtarma yönetimi (KY) olmazsa ne olur?
- Veri Tabanı Kurtarma
 - **KY-1: undo-redo, KY-2:undo-only, KY-3:redo-only**
 - **Pasif, Aktif denetim noktası**
 - **Seyrek Hatalardan Kurtarma**
 - **Kurtalabilirlik Yönünden Plan Çeşitleri (*Recoverable, Cascadeless, Strict*)**
 - **SimpleDB’de KY**

Konu başlıkları

■ EY:

- Serilenebilirlik: (C-S, V-S, DC-S)
- Temel Kilit Protokolü ve 2PL ve versiyonları
- Ölüklilit ve Açlık
- Diğer Kilit Protokolleri (Kilit Yükseltme, Update Kilit, Increment kilit, MGL kilit)
- Kilitler ile Yalıtım seviyelerinin sağlanması
- Phantom
- SimpeDB’de eşzamanlılık kontrolü ve Hareket Yönetimi
- Diğer Eşzamanlılık Protokolleri
 - MV-lock
 - TimeStamp
 - Optimistic methods

Hareket (*transaction, tx*)

- Her istemcinin VTYS'den alacağı servis, bir “**hareket**” kapsamında gerçekleştirilir. **Hareket**, birden çok veri tabanı işleminin bir araya gelmesi ile oluşan program parçacığıdır. Bunun nedeni “**veri bütünlüğü**”nün sağlanmasıdır. Hareket, VT'nı tutarlı bir durumdan başka bir tutarlı bir duruma getirir.
- VTYS hareket yönetimi (HY), çok sayıda istemciden gelen hareketlerin, VT bütünlüğünü (*integrity*) sağlayacak şekilde sonlanmasını sağlamakla yükümlüdür. Bunun için de hareketin, **ACID** özelliklerini sağlaması gerekir.
 - **A**tomicity (atomik, bölünemezlik..): “ya hep ya hiç!”
 - **C**onsistency (tutarlılık): “vt'nını tutarlı bırak!”
 - **I**solation (yalıtım): “sanki bir tek kendisi var!”
 - **D**urability (süreklilik): “onaydan sonra artık sürekli var!”
- Bunlar VTYS'de 2 farklı modül ile sağlanır: **Veri Kurtarma** ve **Eşzamanlılık**
 - Atomik ve Süreklilik, “veri kurtarma” modülünün görevidir ve COMMIT ve ROLLBACK ile sağlanır.
 - Tutarlılık ve Yalıtım, “eşzamanlılık” modülünün görevidir ve LOCK ve benzeri yöntemler ile sağlanır.
- **Kurtarma modülü:**
 - Log kayıtlarının oluşturulması ve diske yazılması
 - Hareketin geretiğinde Geriye dönüşünü (*rollback*) sağlamak
 - Sistemin çökmesinde, veri tabanını kurtarma (*recovery*)
- **Eşzamanlılık modülü:**
 - Birden çok hareketin, aynı anda paralel olarak çalışmasının (sanki seri olarak çalışıp sonlanmış gibi) doğruluğunu sağlamak.

Hareketin durumları

T: r(X);w(X);r(Y);w(Y);

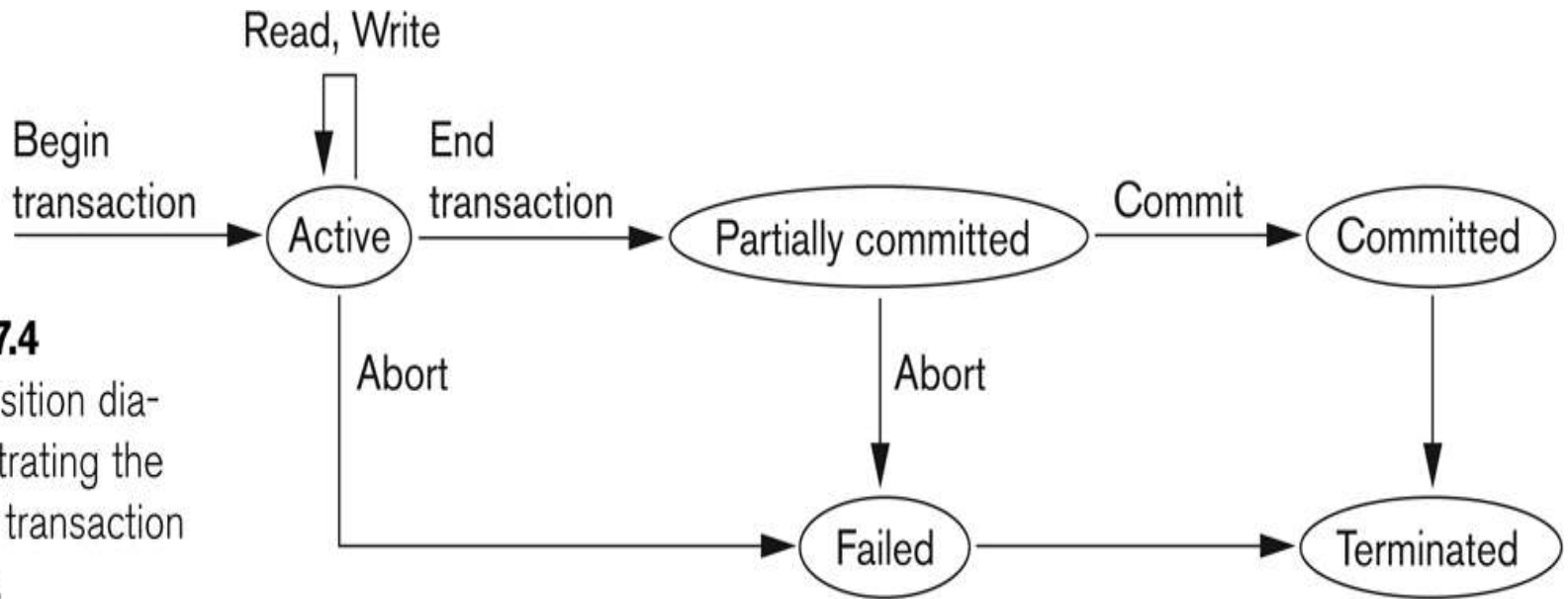


Figure 17.4

State transition diagram illustrating the states for transaction execution.

Kısmi commit durumunda, veri kurtarma yönetimi (KY), hareketin yaptığı değişiklikleri diske kaydetmesinde herhangi bir sistem hatasının sorun çıkarmasına mani olan önlemleri alır. Örneğin, sistem log kayıtlarını öncelikle diske yazar...

HY olmazsa ne olur? -1

```
public void reserveSeat(Connection conn, int custId,
    int flightId) throws SQLException {

    Statement stmt = conn.createStatement();
    String s;

    // Step 1: Get availability and price
    s = "select NumAvailable, Price from SEATS "
        + "where FlightId = " + flightId;
    ResultSet rs = stmt.executeQuery(s);
    if (!rs.next()) {
        System.out.println("Flight doesn't exist");
        return;
    }
    int numAvailable = rs.getInt("NumAvailable"); R(A)
    int price = rs.getInt("Price"); R(B)
    rs.close();

    if (numAvailable == 0) {
        System.out.println("Flight is full");
        return;
    }

    // Step 2: Update availability W(A)
    int numseats = numAvailable - 1;
    s = "update SEATS set NumAvailable = " + numseats
        + " where FlightId = " + flightId;
    stmt.executeUpdate(s);

    // Step 3: Get and update customer balance
    s = "select BalanceDue from CUST "
        + "where CustID = " + custId; R(C)
    rs = stmt.executeQuery(s);
    int newBalance = rs.getInt("BalanceDue") + price;
    rs.close();

    s = "update CUST set BalanceDue = " + newBalance
        + " where CustId = " + custId; W(C)
    stmt.executeUpdate(s);
}
```

Figure 14-1
JDBC code to reserve a seat on a flight

Bir uçak rezervasyon sistemine ait veri tabanı şeması: (*KOLTUKLAR* tablasunda, her uçuş için boş koltuk sayısı ve koltuk ücreti tutuluyor. *MÜŞTERİ* tablosunda ise, her müşterinin ödemesi gereken miktar bilgisi saklanmaktadır.)

- KOLTUKLAR(UçuşNo, BoşKoltukSayısı, Ücret)
- MÜŞTERİ(MüşteriNo, Borc)

Aşağıdaki 3 farklı senaryoyu inceleyin:

1. A ve B olarak 2 farklı müşteri bu programı çalıştırıyorlar. A müşterisi 1. adım sonunda iş kesme (interrupt) ile beklemeye başladı. B müşterisi programı çalıştırıp sonlandırdı. A müşterisi kaldığı yerden devam edip programı sonlandırdı.
2. Sadece A müşterisi programı çalıştırıyor. Programın 2. adım sonlandıktan sonra, veri tabanı yönetim sistemi çöküyor.
3. Sadece A müşterisi programı çalıştırıyor. Programın 3. adımı sonlandıktan sonra, veri tabanı yönetim sistemi çöküyor.

HY olmazsa ne olur? -2 (***autocommit (true)***)

- **'autocommit(true)'** ile, JDBC hareket işlemeyi kullanıcıdan gizleyebilir. Ancak, her SQL komutu bir hareket gibi işlenir.
 - *executeupdate(...)* yenileme gerçekleşince sonlanır.
 - *executequery(...)* ise, kullanıcı sorgu sonuç setini kapatınca, *rs.close()* veya yeni bir sorgu çalışmaya başlayınca sonlanır.
- Hareket, sonlanıncaya kadar sahip olduğu kaynakları (kilitler, tablolar..) tutar; bu sistemin başka hareketlere olan hizmetini yavaşlatır. Onun için;
 - Kısa hareketler tercih edilir.
 - Kullanıcı, programda işi bitince hareketi *rs.close()* ile sonlandırması gerekir.
 - **'autocommit(true)'**, tercih edilir. Zaten bu durum default..
- Açıktan bir HY olmaması durumu olan **'autocommit (true)'** durumu ne zamanlarda yetersiz kalır?
 - 1) Birden çok kullanıcının (hareketin) aktif olmasının gerektiği durumlar.
 - 2) Veritabanında birden çok değişikliğin aynı anda olması durumları.
 - 3) *Bulk loading* denilen, bilginin dış ortamdan çok sayıda insert işlemiyle vt'ye aktarılması.

HY olmazsa ne olur? -3 (*autocommit (true)*)

```
DataSource ds = ...
Connection conn = ds.getConnection();
Statement stmt1 = conn.createStatement();
Statement stmt2 = conn.createStatement();
String qry = "select * from COURSE";
ResultSet rs = stmt1.executeQuery(qry);
while (rs.next()) {
    String title = rs.getString("Title");
    boolean goodCourse = getUserDecision(title);
    if (!goodCourse) {
        int id = rs.getInt("CId");
        stmt2.executeUpdate("delete from COURSE "
                           + "where CId =" + id);
    }
}
rs.close();
```

```
DataSource ds = ...
Connection conn = ds.getConnection();
Statement stmt = conn.createStatement();
String cmd1 = "update SECTION set Prof= 'brando' "
              + "where SectId = 43";
String cmd2 = "update SECTION set Prof= 'einstein' "
              + "where SectId = 53";

stmt.executeUpdate(cmd1);
// suppose that the server crashes at this point
stmt.executeUpdate(cmd2);
```

- Yukarıdaki 1) ve 2) durumlarına karşılık gelen örnek senaryoların doğru çalışması için kullanıcının, **Connection** sınıfında hareketin **autocommit** modunu **false** yapması ve bu yüzden program içerisinde **commit** ve **rollback** yapması gerekir...JDBC Connection sınıfının bununla ilgili kısmı:

```
public void setAutoCommit(boolean ac) throws SQLException;
public void commit() throws SQLException;
public void rollback() throws SQLException;
```

Figure 8-11

The *Connection* API for handling transactions explicitly

Önceki sunuda 1) numaralı programın doğru hali

```
DataSource ds = ...
Connection conn = ds.getConnection();
Statement stmt1 = conn.createStatement();
Statement stmt2 = conn.createStatement();
String qry = "select * from COURSE";
ResultSet rs = stmt1.executeQuery(qry);
while (rs.next()) {
    String title = rs.getString("Title");
    boolean goodCourse = getUserDecision(title);
    if (!goodCourse) {
        int id = rs.getInt("CId");
        stmt2.executeUpdate("delete from COURSE "
            + "where CId =" + id);
    }
}
rs.close();
```

1)

```
Connection conn = null;
try {
    DataSource ds = ...
    Connection conn = ds.getConnection();
    conn.setAutoCommit(false);

    Statement stmt = conn.createStatement();
    String qry = "select * from COURSE";
    ResultSet rs = stmt.executeQuery(qry);
    while (rs.next()) {
        String title = rs.getString("Title");
        boolean goodCourse = getUserDecision(title);
        if (!goodCourse) {
            int id = rs.getInt("CId");
            String cmd = "delete from COURSE"
                + "where CId =" + id;
            stmt.executeUpdate(cmd);
        }
    }
    rs.close();
    conn.commit();
} catch (SQLException e) {
    try {
        e.printStackTrace();
        conn.rollback();
    }
    catch (SQLException e2) {
        System.out.println("Could not roll back");
    }
}
finally {
    try {
        if (conn != null)
            conn.close();
    }
    catch (Exception e) {
```

SimpleDB'de Hareket Kullanımı

```
public void    commit();  
public void    rollback();  
public void    recover();
```

```
public void    pin(Block blk);  
public void    unpin(Block blk);  
public int     getInt(Block blk, int offset);  
public String  getString(Block blk, int offset);  
public void    setInt(Block blk, int offset, int val);  
public void    setString(Block blk, int offset, String val);
```

```
public int     size(String filename);  
public Block   append(String filename, PageFormatter fmtr);
```

Figure 14-2

The API for the SimpleDB class *Transaction*

- Her bir JDBC istemci için bir Transaction sınıfı vardır.
- 1. kısım: hareketin sonlanması, geri sarması ve vt kurtarma
- 2. kısım: burada amaç tampon kullanımının kullanıcıdan gizlenmesidir. Örneğin, *setInt(...)*, istenilen bloğa karşılık gelen tampona erişmek için **gerekli kilitleri alır** ve yapılan değişikliklere ait **log kayıtları** oluşturur. Demek ki, **hareket** tampon kullanımını kullanıcıdan gizler; böylece eşzamanlılığı, yalıtımı ve kayıt defteri oluşturmayı da kullanıcıdan gizler.
- 3.kısım: dosya sistemi ile ilgili metodlardır. Bu metodlar da, tutarlılık için kilit kullanılmalıdır. (*Phantom probleminin olmaması için...*)

SimpleDB'de bir "hareket" uygulaması

```
SimpleDB.init("studentdb");
Transaction tx = new Transaction();
Block blk = new Block("student.tbl", 0);
tx.pin(blk);
int gy = tx.getInt(blk, 38);
String sname = tx.getString(blk, 46);
tx.unpin(blk);
System.out.println(sname + "\t" + gy);

tx.pin(blk);
n = tx.getInt(blk, 38); //Do we need this stmt?
tx.setInt(blk, 38, n+1);
tx.unpin(blk);

PageFormatter pf = new ABCStringFormatter();
Block newblk = tx.append("junk", pf);
tx.pin(newblk);
String s = tx.getString(newblk, 0); //will be "abc"
tx.unpin(newblk);
int blknum = newblk.number();
System.out.println("The first value in block "
                  + blknum + " is " + s);

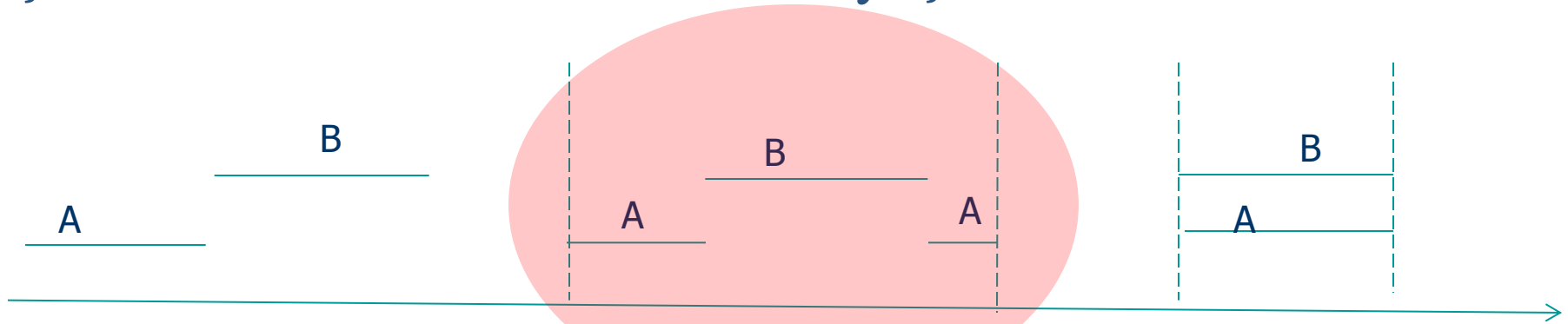
tx.commit();
```

Figure 14-3

Using a SimpleDB transaction

- Bir hareket içerisinde bir değeri okuma işlemi niye tekrar etsin ki?
- Şekildeki sorunun cevabı: Gerek yok. Çünkü tx içerisinde olduğu için başka tx', tx bitmeden bu değeri gelip de değiştiremez.
 - Değiştirememesi ümit edilir..
 - Farklı yalıtım derecelerine göre durum değişebilir...

Eşzamanlılık Ne Demek? Ne ihtiyaç var?



Daha çok eşzamanlılık (daha yüksek performans)

- Yüksek kaynak (cpu,disk) kullanım verimliliği (better resource utilization)
- Sistem tx hizmet kapasitesi (high system throughput)
- Ortalama sistem cevap süresinin azalması (better ave. response time)

Bazı Tanımlar:

Veri tabanı eylemi (db action): bir bloğun diskten okunması-yazılması

R(X): X bloğunun diskten tampona yazılması ve okunması

W(X): X bloğunun diskten tampona yazılması, değişiklik yapılması ve sonra hemen veya bir zaman sonra diske geri yazılması

Tarihçe: bir tx'nın vt dosyaları üzerindeki erişim dizisi. Örneğin:

tx: R(student.tbl,0); R(student.tbl,0); W(student.tbl,0);

Plan: vt üzerinde işlem yapan **bütün tx'ların (öngörülemez halde)** içiçe girmiş tarihçeleri.

Örnek Plan (*schedule*)

(a) T_1

```
read_item (X);  
X:=X-N;  
write_item (X);  
read_item (Y);  
Y:=Y+N;  
write_item (Y);
```

(b) T_2

```
read_item (X);  
X:=X+M;  
write_item (X);
```

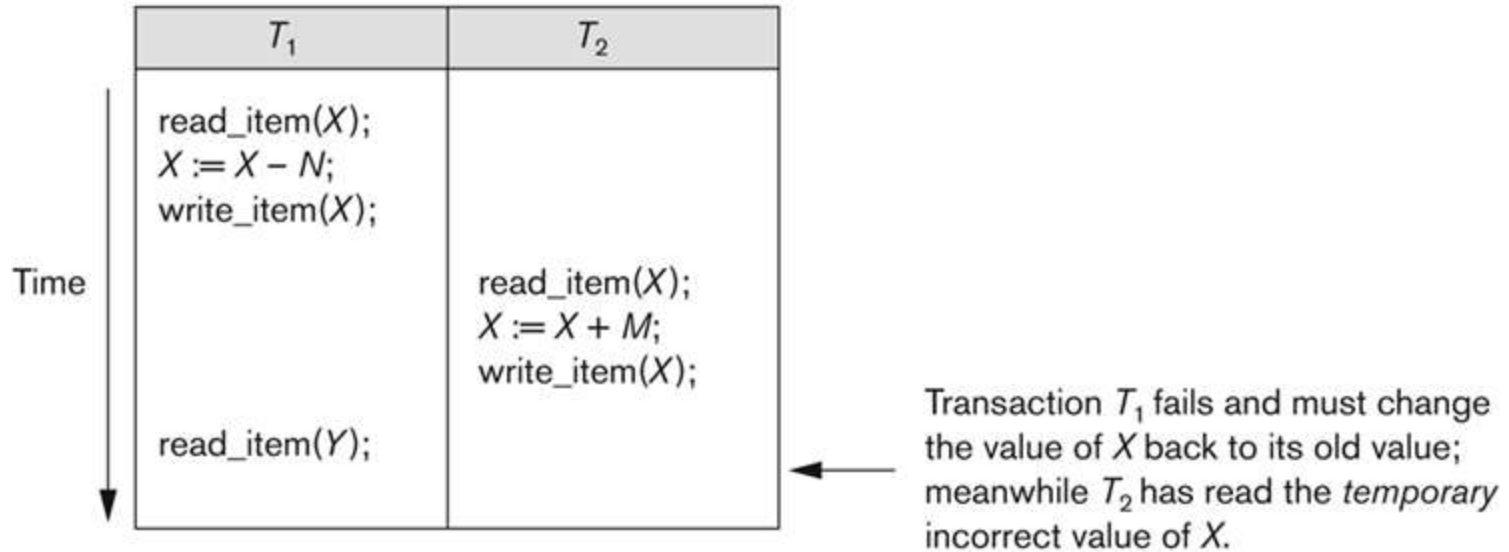
- $T_1: R_1(X), W_1(X), R_1(Y), W_1(Y)$
- $T_2: R_2(X), W_2(X)$
- Farklı plan sayısı toplam $(4+2)! / (4! * 2!) = 6*5*4*3*2*1 / 4*3*2*1*2*1 = 15$.
- Farklı SERİ planların sayısı : $2!=2$
- PLAN 1: $r_1(X); w_1(X); r_1(Y); r_2(X); w_1(Y); w_2(X);$
- PLAN 2: $r_2(X); r_1(X); w_1(X); r_1(Y); w_2(X); w_1(Y);$
-
- PLAN 15: ...

Eşzamanlılık Yön. (EY) olmazsa ne olur?

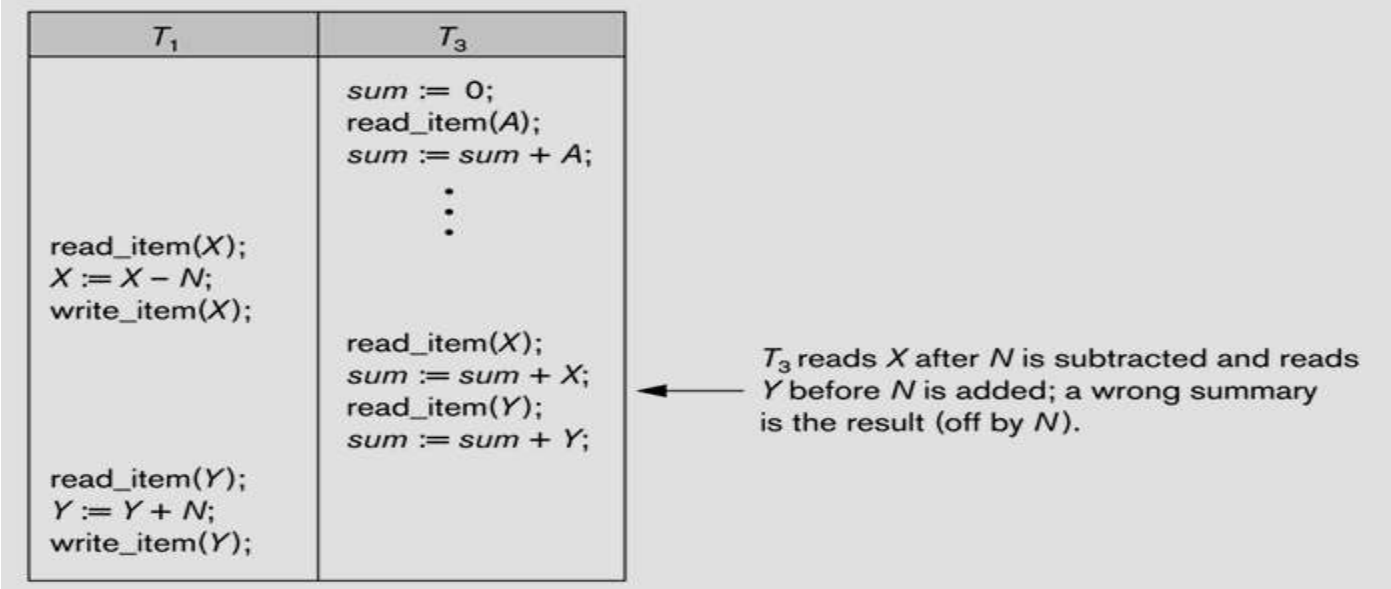


Hareket, vt'nı tutarlı bir durumdan başka bir tutarlı duruma taşıyor. Bu geçişte, vt tutarsız durumlardan geçebilir. Bu geçişteki tutarsız durumların yalıtılması; yani diğer hareketlere yansıtılmaması gerekiyor. Yansıtılmaması miktarı **yalıtım seviyesinin** set edilmesi ile oluyor.

Eşzamanlılık Yön. (EY) olmazsa ne olur? – *dirty read*



Eşzamanlılık Yön. (EY) olmazsa ne olur? *incorrect summary*



```
Connection conn = ds.getConnection();
conn.setAutoCommit(false);
Statement stmt = conn.createStatement();
String cmd1 = "update SECTION set Prof= 'brando' "
              + "where SectId = 43";
String cmd2 = "update SECTION set Prof= 'einstein' "
              + "where SectId = 53";

stmt.executeUpdate(cmd1);
stmt.executeUpdate(cmd2);
```

Yandaki H1 hareketi, her öğretmenin kaç ders verdiğini hesaplayan H2 hareketi ile beraber çalışsın..

Eşzamanlılık Yön. (EY) olmazsa ne olur?—*lost update*

```
DataSource ds = ...
Connection conn = ds.getConnection();
conn.setAutoCommit(false);
Statement stmt = conn.createStatement();
String qry = "select MealPlanBalance "
            + "from STUDENT where SId = 1";
ResultSet rs = stmt.executeQuery(qry);

rs.next();
int balance = rs.getInt("MealPlanBalance");
rs.close();

int newbalance = balance - 10;
if (newbalance < 0)
    throw new NoFoodAllowedException();

String cmd = "update STUDENT "
            + "set MealPlanBalance = " + newbalance
            + " where SId = 1";
stmt.executeUpdate(cmd);

conn.commit();
```

(a) Transaction T_1 decrements the meal plan balance

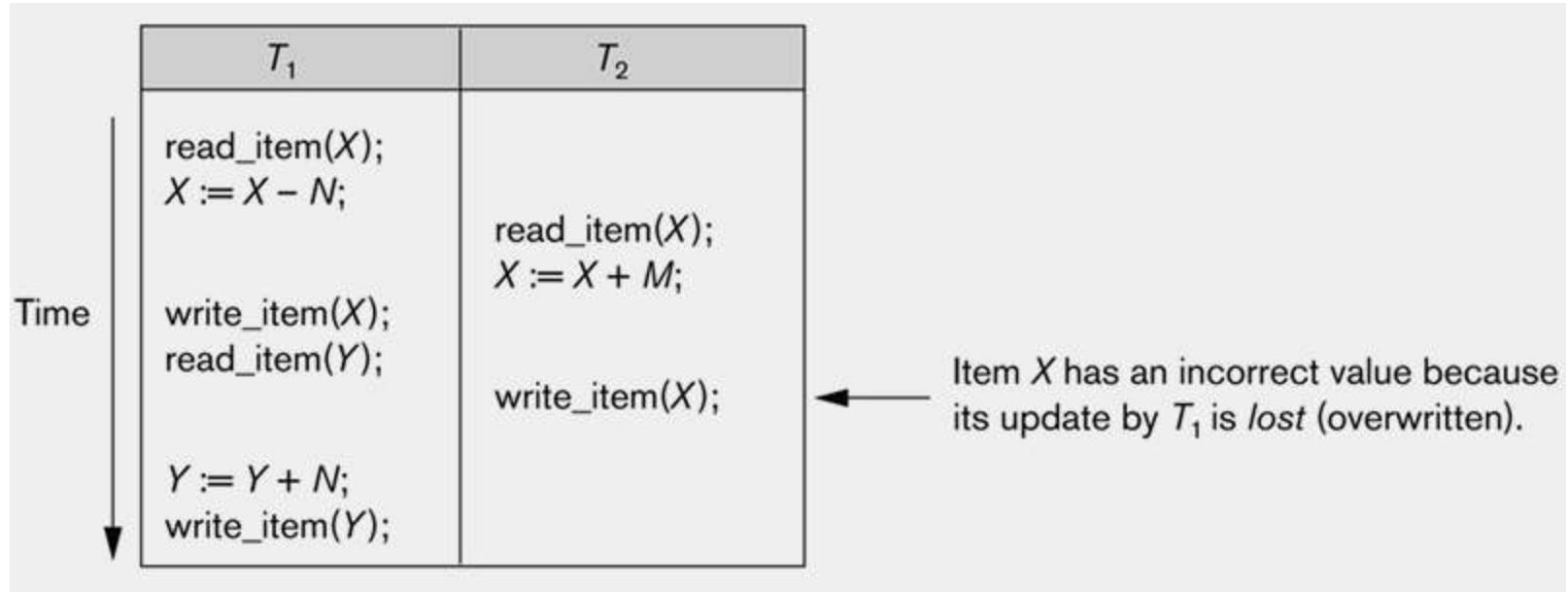
```
DataSource ds = ...
Connection conn = ds.getConnection();
conn.setAutoCommit(false);
Statement stmt = conn.createStatement();
String cmd = "update STUDENT "
            + "set MealPlanBalance = MealPlanBalance + 1000"
            + " where SId = 1";
stmt.executeUpdate(cmd);

conn.commit();
```

(b) Transaction T_2 increments the meal plan balance

Soldaki T_1 hareketi, **balance** değerini okumasıyla, değiştirmesi arasında **balance**'in değişmediğini varsayıyor...Oysa bu bu ancak *repetable read* yalıtımında mümkün...

Eşzamanlılık Yön. (EY) olmazsa ne olur?—*lost update*



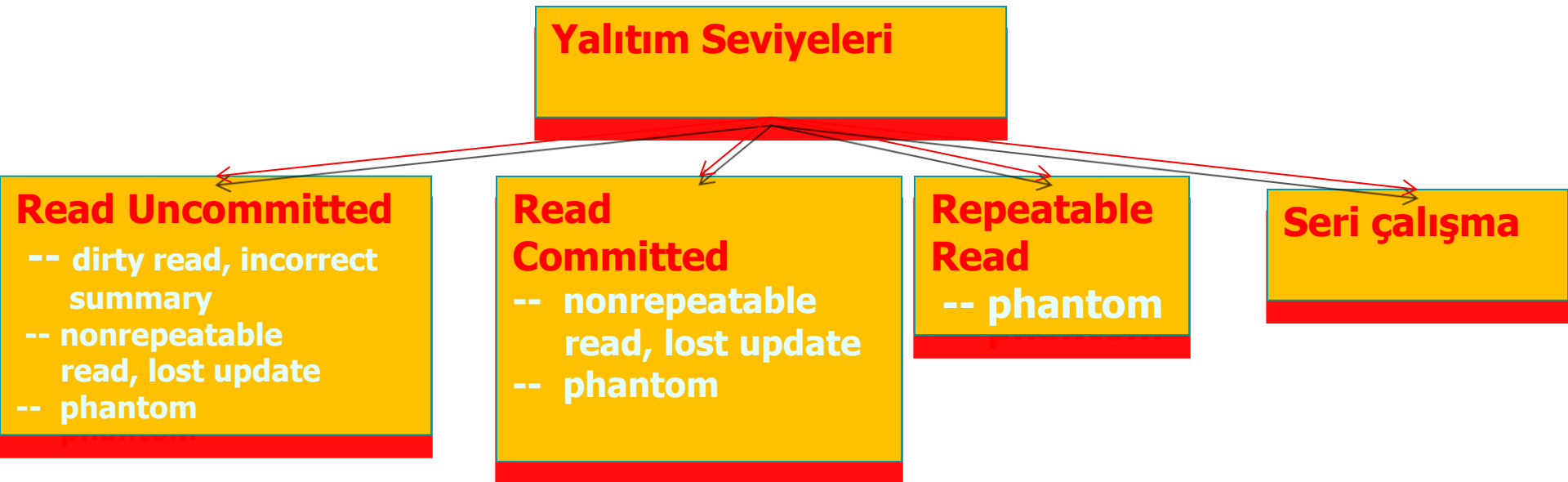
Eşzamanlılık Yön. (EY) olmazsa ne olur? -*phantom problem*

```
DataSource ds = ...
Connection conn = ds.getConnection();
conn.setAutoCommit(false);
Statement stmt = conn.createStatement();
String qry = "select count(SId) as HowMany "
    + "from STUDENT "
    + "where GradYear >= extract(year, current_date)";
ResultSet rs = stmt.executeQuery(qry);
rs.next();
int count = rs.getInt("HowMany");
rs.close();

int rebate = 100000 / count;
String cmd = "update STUDENT "
    + "set MealPlanBalance = MealPlanBalance + "
    + rebate
    + " where GradYear >= extract(year, current_date)";
stmt.executeUpdate(cmd);
conn.commit();
```

- Yukarıdaki H1 hareketi, yeni öğrenci ekleyen başka bir H2 hareketi ile beraber çalışsın...

Yalıtım seviyeleri



- Çoğu veri tabanında *default* yalıtım seviyesi: **read committed** ...Yani, bir hareket, başka bir hareket tarafından **değiştirilmiş ve commit edilmemiş** bir bilgiye erişemez.. Bu, **dirty read, incorrect summary** problemlerini çözer..
- Bir hareket, okuduğu bir değer başkaları tarafından değiştirmesini farkedemiyorsa bu bir **nonrepeatable read** problemidir. **Lost update** buna bir örnektir. Bunu çözen yalıtım seviyesi **repeatable read** dir.
- H1 hareketi tabloda seçtiği kayıtlar üzerinde bir işlem gerçekleştiriyorsa; ve bu arada diğer bir hareket H1'in seçim kriterine uygun yeni kayıtlar ekliyorsa bu bir **phantom** problemidir. Bunu çözen yalıtım ancak **seri çalışmadır**.

Kurtarma yönetimi (KY) olmazsa ne olur?-

- Çünkü: Hareketin **A** ve **D** özelliğini sağlamak için..
- SİSTEM KURTARMA, bir hareketin herhangi bir tipik hatada «geri sarılamaması veya bütün yaptığı değişikliklerin diske kaydedilememesi» durumlarına mani olacak önlemlerin alınmasını sağlar. (*WAL, Partially commit gibi*)
- Tipik hatalar: (*log veya shadow yöntemleri ile kurtarılabilenler*)
 - Sistem kilitlenmesi (*örn. Ana hafıza hataları..*)
 - Hareket veya sistem hatası (*örn. hareket içi mantıksal hatalar, 0 ile bölme ..*)
 - Yerel hatalar veya hareketin tespit ettiği özel durumlar (*hareketin ulaşmak istediği bilginin bulunamaması veya yetersizliği*)
 - Eşzamanlılık kontrol mekanizasının zorlaması (*deadlock, serilenebilirlik ihlali gibi..*)
- Seyrek hatalar: (*ancak VT arşiv/backup ile kurtarılabilenler*)
 - Disk hataları (*disk kolu hataları*)
 - Fiziksel veya dışsal felaketler.. (yangın, sel ..ve diğerleri)

Kurtarma Yönetimi

- **KY:**

- **KY-1: undo-redo, KY-2:undo-only, KY-3:redo-only**
- Pasif, Aktif denetim noktası
- Seyrek Hatalardan Kurtarma
- Kurtalabilirlik Yönünden Plan Çeşitleri (*Recoverable, Cascadeless, Strict*)
- SimpleDB'de KY

Kurtarma Yönetimi (KY)

■ Log kayıtlarını yazmak:

- start, commit, rollback, update ve append (*fakat append SimpleDB'de henüz yok!*)
- Bir harekete ait olan log kayıtları log dosyasında birbirine bitişik (*contiguous*) mi?

```
<START, 27>  
<SETINT, 27, student.tbl, 0, 38, 2005, 2006>  
<COMMIT, 27>
```

Figure 14-4

The log records generated from Figure 14-3

■ Tx'i geri sarma:

1. Set the current record to be the most recent log record.
2. Do until the current record is the start record for T:
 - a) If the current record is an update record for T then:
Write the saved old value to the specified location.
 - b) Move to the previous record in the log.
3. Append a rollback record to the log.

Figure 14-5

The algorithm for rolling back transaction T

- vt'ni kurtarma: vt her yeniden başlamasında mutlaka çalışır; amacı vt'ni tutarlı bir duruma getirmektir. Önceki kapanma (*shut down*) normal ise bir şey yapmaz; eğer sistem kilitlenip kapandıysa sistemi tutarlı duruma taşınmalı..

■ Tutarlı durum:

- tamamlanmamış tx'ları geri sarmalı (**undo**)
- bütün tamamlanmış (commit edilmiş) tx'lere ait tamponlar diske yazılmalı (**redo**)

Neyi Kurtarıyoruz?

Veri taneselliği (*data granularity*)

- Tanesellik: log kaydına sebep olacak minimum veri parçası (*data item*) büyüklüğüdür.
- Bir log kaydı, taneselliğe göre kaydın içinde veri parçasının eski yeni halini saklar.
- Tanesellik:
 - Değer: her değer değişimi → 1 log kaydı
 - Blok: her blok değişimi → 1 log kaydı.
 - Eğer bir blok üzerinde çok değişiklik yapılıyorsa kullanılır
 - Dosya: her dosya değişiliği → 1 log kaydı
 - Vt için uygun olmasa da; başka uygulamalarda kullanılıyor.(MSword gibi)

Hareketin COMMIT noktası

- Hareketin VT eylemlerini başarı ile tamamladığı ve ilgili LOG kayıtlarını log dosyasına(diske) yazması ve ardından $\langle \text{COMMIT}, T \rangle$ log kaydını log dosyasına(diske) yazmasıdır.
- COMMIT: bir anlaşma, bir vaad.

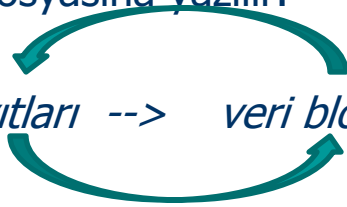
WAL (write ahead logging)

- bir veri tamponundaki değişikliğe ait log kaydının diske yazılması; bu veri tamponunun diske yazılmasından **önce** gerçekleşmesi gerekir. *(Bkz. Sunu 3, sf:14, class Buffer)*
- WAL kuralı sayesinde; veri tamponundaki değişikliğin flush edilmesi ve diskte olması, oysa ilgili log kaydının olmaması gibi **istenmeyen bir durum** söz konusu olamaz..*(Diğer 3 farklı durum sözkonusu olabilir; ki bunların zararı yoktur)*

Vt kurtarma 1: **undo-only**

LOG KURALLARI:

1. $\langle T, X, v \rangle$ log kaydında; v , X 'in eski değeri.
2. WAL
3. Hareketin değişiklik yaptığı veri blokları diske yazıldıktan hemen sonra (en kısa zamanda) COMMIT log kaydı log dosyasına yazılır.



(Diske yazma sırası: log kayıtları --> veri blokları --> COMMIT log kaydı)

KURTARMA YÖNTEMİ:

1. Log dosyasında **sondan başa doğru** gidilirken $\langle \text{COMMIT } T \rangle$ olan T hareketleri C-listesine eklenir.
2. Karşılaşılan $\langle T, X, v \rangle$ log kaydındaki T ,
 - C-listesinde DEĞİLSE undo yapılır. (Demek ki, T ya aktif ya da abort olmuş bir harekettir).
 - C-listesinde ise herhangi bir işlem yapılmaz.

SONUÇ:

- KURTARMA HIZLI; çünkü sadece undo
- log dosyaları daha küçük; çünkü yeni değeri log dosyasına yazmaya gerek yok.
- COMMIT DAHA YAVAŞ; çünkü tx'a ait bütün tamponların flush edilmesini bekliyor.. Bu da disk yazma sayısında genel olarak artışa sebep olur...
- Sistem çökmesinin sık olduğu durumlarda tercih edilebilir..
- Gerek UNDO-ONLY, (gerek UNDO-REDO için), COMMIT log kaydı diske ulaşmadan sistem kilitlenmesi mümkün. Bu durumda her iki algoritma da, bu hareketi undo yapmaya mecbur kalıyor. (Bu olasılık UNDO-REDO'da daha düşük...)

Vt kurtarma 3: redo-only

LOG KURALLARI:

1. $\langle T, X, v \rangle$ log kaydında; v , X 'in yeni değeri.
2. WAL
3. Veri bloklarındaki değişiklikler, ancak COMMIT log kaydı log dosyasına yazıldıktan sonra diske yazılmaya başlanır.

(Diske yazma sırası: log kayıtları --> COMMIT log kaydı --> veri blokları)

KURTARMA YÖNTEMİ:

1. Log dosyasında **sondan başa doğru** gidilirken $\langle \text{COMMIT } T \rangle$ olan T hareketleri C-listesine eklenir.
2. Log dosyasında **başdan sona doğru** gidilirken karşılaşılan $\langle T, X, v \rangle$ log kaydındaki T ,
 - C-listesinde ise redo yapılır.
 - C-listesinde değil ise $\langle \text{ABORT } T \rangle$ log kaydını log dosyasına flush et..

SONUÇ:

- KURTARMA HIZLI; çünkü sadece redo
- Veri bloklarının tampon havuzunda daha uzun süre kalabilmelerini sağlayarak disk erişim verimliliği artması ümid edilir.
- Tampon mücadelesi daha yoğun; çünkü tamponu yakalayan bir tx bırakmıyor. Bu da TPM (tx per minute) değerini düşürüyor.
- Küçük hareketler ve/veya az değişiklik yapılan hareketlerde tercih edilir.

Vt kurtarma 1: **undo-redo**

LOG KURALLARI:

1. $\langle T, X, v, w \rangle$ log kaydında; v , X 'in eski değeri; w ise yeni değeri
2. WAL



(Diske yazma sırası: log kayıtları --> veri blokları --> **COMMIT log kaydı** --> veri blokları)

KURTARMA YÖNTEMİ:

1. Log dosyasında sondan başa doğru gidilirken $\langle \text{COMMIT } T \rangle$ olan T hareketleri C-listesine eklenir.
 - karşılaşılan $\langle T, X, v \rangle$ log kaydındaki T , C-listesinde değil ise undo yapılır.
2. Log dosyasında başdan sona doğru gidilirken
 - karşılaşılan $\langle T, X, v \rangle$ log kaydındaki T , C-listesinde ise redo yapılır.

SONUÇ:

- KURTARMA YAVAŞ.
- Diske yazma sırası çok esnek: (*eşzamanlılık taneselliği blok'tan daha küçük olduğunda Undo veya Redo 'da ortaya çıkabilecek çelişki sorunları burada yok.*)
- Sistem disk erişim performansı daha iyi.
- Tampon kullanımı daha verimli.
- *NOT: COMMIT log kaydını diske yazmakta acele etmek gerekiyor. Çünkü COMMIT log kaydı tampona yazılsa fakat diske ulaşmasa sistem kilitletmesinde hareket gereksiz yere UNDO olacak..*

Örnek (undo-only)

Undo-only LOG DOSYASI:

1. <START T1>
2. <T1,A,5>
3. <START T2>
4. <T2,B,10>
5. <T2,C,15>
6. <START T3>
7. <T1,D,20>
8. <COMMIT T1>
9. <T3,E,25>
10. <COMMIT T2>
11. <T3,F,30>
- 12.

Sistem ilk çalıştığında log dosyasında görünen son satır:

- 11.satır ise;
 - undo T3: F=30; E=25;
- 9.satır ise;
 - undo T2: C=15; B=10;
 - undo T3: E=25
- 7.satır ise;
 - Undo T1: D=20; A=5
 - undo T2: C=15; B=10;

Örnek: *redo-only*

T_1
read_item(A)
read_item(D)
write_item(D)

T_2
read_item(B)
write_item(B)
read_item(D)
write_item(D)

[start_transaction, T_1]
[write_item, T_1 , D, 20]
[commit, T_1]
[start_transaction, T_2]
[write_item, T_2 , B, 10]
[write_item, T_2 , D, 25]

← System crash

- Redo-only kurtarma (tek hareket):
 - Log kayıtlarında 2 tür hareket kaydı var: Commit edilmişler, 1 tane aktif.
 - Denetim noktasına kadar karşılaşılan Commit edilmişler redo yapılmalı, aktif olan harekete herhangi bir işlem yapılmamalı.
- Örnekte, sadece T1 redo yapılmalı. D=20 tekrar çalıştırılmalı.

Örnek (undo-redo)

Undo-Redo LOG DOSYASI:

1. <START T1>
2. <T1,A,5,6>
3. <START T2>
4. <T2,B,10,11>
5. <T2,C,15,16>
6. <START T3>
7. <T1,D,20,21>
8. <COMMIT T1>
9. <T3,E,25,26>
10. <COMMIT T2>
11. <T3,F,30,31>
- 12.

Sistem ilk çalıştığında log dosyasında görünen son satır:

- 11.satır ise;
 - Undo T3: F=30; E=25;
 - Redo T1: A=6, D=21
 - Redo T2: B=11, C=16
- 9.satır ise;
 - Undo T2: C=15; B=10;
 - Undo T3: E=25
 - Redo T1: A=6, D=21
- 7.satır ise;
 - Undo T2: C=15; B=10;
 - Undo T3: --
 - Undo T1: A=5, D=20

Kurtarma Yönetimi

- **KY:**
 - KY-1: undo-redo, KY-2:undo-only, KY-3:redo-only
 - Pasif, Aktif denetim noktası
 - Seyrek Hatalardan Kurtarma
 - Kurtalabilirlik Yönünden Plan Çeşitleri (*Recoverable, Cascadeless, Strict*)
 - SimpleDB'de KY

Pasif denetim noktası (*quiescent checkpoint*)

**Pasif CKPT
protokolü**



1. Stop accepting new transactions.
2. Wait for existing transactions to finish.
3. Flush all modified buffers.
4. Append a quiescent checkpoint record to the log and flush it to disk.
5. Start accepting new transactions.

- Log dosyasındaki “Denetim noktası” (CKPT)
 - Bu noktadan önceki bütün log kayıtları, commit edilmiş ve tamponları diske yazılmış tx'lara ait olan log kayıtlarıdır.

KURTARMA YÖNTEMİ: (*undo-only, redo-only ve undo-redo için geçerli*)

- KY, checkpoint kaydını görünce durur. (*log dosyasının başına kadar okumaya gerek yok*). Bundan sonra; (*eğer redo-only veya undo-redo ise*) 2.adım olan redo yapmaya başlar.
- KY, herhangi bir zamanda **sessiz** denetim noktası oluşturabilir. Belirli periyotlar ile veya Kurtarma işlemi sonunda denetim noktası oluşturmak mantıklı!

**Örnek:
Pasif CKPT içeren
Undo-redo log
dosyası**

```
<START, 0>
<SETINT, 0, student.tbl, 0, 38, 2004, 2005>
<START, 1>
<START, 2>
<COMMIT, 1>
<SETSTRING, 2, junk, 44, 20, hello, ciao>
//The quiescent checkpoint procedure starts here
<SETSTRING, 0, student.tbl, 0, 46, amy, aimee>
<COMMIT, 0>
//tx 3 wants to start here, but must wait
<SETINT, 2, junk, 66, 8, 0, 116>
<COMMIT, 2>
<CHECKPOINT>
<START, 3>
<SETINT, 3, junk, 33, 8, 543, 120>
```

Aktif denetim noktası (*nonquiescent checkpoint*)

- Pasif denetim noktasında sistem bir süre hizmet dışı oluyor.
- Aktif denetim noktası (*fuzzy CKPT*); devam etmekte olan tx'ların bitmesini beklemeden 2 aşamalı CKPT'dir. AMAÇ: sistemin devre dışı olma süresini azaltmak.

UNDO-ONLY'de AKTİF CKPT PROTOKOLÜ:

- $\langle \text{START CKPT } (T_1, \dots, T_k) \rangle$ log kaydını log dosyasına yaz. $T_1, \dots, T_k$, CKPT başladığı zaman aktif olan (*commit olmamış olan*) hareketlerdir.
- $T_1, \dots, T_k$ hareketlerinin hepsi COMMIT oluncaya kadar bekle. **Bu arada yeni ortaya çıkan hareketlere mani olma.**
- $\langle \text{END CKPT} \rangle$ log kaydını dosyaya yaz.

KURTARMA YÖNTEMİ:

Aynı undo-only gibi. Fakat, Nerede duracağız?

- Eğer $\langle \text{END CKPT} \rangle$ gördüysek; $\langle \text{START CKPT } (T_1, \dots, T_k) \rangle$ log kaydına kadar gideriz.
- Eğer $\langle \text{END CKPT} \rangle$ görmeden, $\langle \text{START CKPT } (T_1, \dots, T_k) \rangle$ gördüysek; en eski $\langle \text{START } T_i \rangle$, $i: 1, \dots, k$ log kaydını görünceye kadar gideriz.

Örnek: undo-only'de CKPT

1. <START T1>
2. <T1,A,5>
3. <START T2>
4. <T2,B,10>
5. **<START CKPT (T1,T2)>**
6. <T2,C,15>
7. <START T3>
8. <T1,D,20>
9. <COMMIT T1>
10. <T3,E,25>
11. <COMMIT T2>
12. **<END CKPT>**
13. <T3,F,30>

Sistem ilk çalıştığında log dosyasında görünen son satır:

- 13.satır ise;
 - undo T3: F=30; E=25;
 - 5.satırı görünce dur.
- 10.satır ise;
 - undo T3: E=25
 - undo T2: C=15; B=10;
 - 3.satırı görünce dur.

Aktif denetim noktası (*nonquiescent checkpoint*)

REDO-ONLY'de AKTİF CKPT PROTOKOLÜ:

- $\langle \text{START CKPT } (T_1, \dots, T_k) \rangle$ log kaydını log dosyasına yaz. $T_1, \dots, T_k$, CKPT başladığı zaman aktif olan (commit olmamış olan) hareketlerdir.
- $\langle \text{START CKPT } (T_1, \dots, T_k) \rangle$ diske yazıldıktan sonra bu ana kadar **COMMIT olmuş olan hareketlere ait kirli tamponları** diske yaz. **Bu arada yeni ortaya çıkan hareketlere mani olma.**
- $\langle \text{END CKPT} \rangle$ log kaydını diske yaz.

KURTARMA YÖNTEMİ:

Aynı redo-only gibi. Fakat, REDO'ya başlama noktasını bulmam gerek? Sondan başa doğru giderken;

- Eğer $\langle \text{END CKPT} \rangle$ gördüysek; $\langle \text{START CKPT } (T_1, \dots, T_k) \rangle$ log kaydındaki önceki **en eski $\langle \text{START } T_i \rangle$, $i: 1, \dots, k$** log kaydını görünceye kadar gideriz. Bundan sonra ileriye doğru giderken görünen $\langle T, X, v \rangle$ de T , C-listesindeyse redo yapılır. Değilse $\langle \text{ABORT } T \rangle$ yazılır.
- Eğer $\langle \text{END CKPT} \rangle$ görmeden, $\langle \text{START CKPT } (T_1, \dots, T_k) \rangle$ gördüysek; evvelki $\langle \text{END CKPT} \rangle$ ve ona ait olan $\langle \text{START CKPT } (S_1, \dots, S_m) \rangle$ log kaydına kadar gideriz. Bundan sonra ileriye doğru giderken görünen $\langle T, X, v \rangle$ de T , C-listesindeyse redo yapılır. Değilse $\langle \text{ABORT } T \rangle$ yazılır.

Örnek: redo-only'de CKPT

Sistem ilk çalıştığında log dosyasında görünen son satır:

1. <START T1>
2. <T1,A,5>
3. <START T2>
4. <COMMIT T1>
5. <T2,B,10>
6. <START CKPT (T2)>
7. <T2,C,15>
8. <START T3>
9. <T3,D,20>
10. <END CKPT>
11. <COMMIT T2>
12. <COMMIT T3>

- 12.satır ise;
 - T1 değişiklikleri diske yazılmış. 3. satır redo başlama noktası.
 - C-listesi: {T2 ve T3}
 - redo T2: B=10; C=15;
 - Redo T3: D=20
- 11.satır ise;
 - T1 değişiklikleri diske yazılmış. 3. satır redo başlama noktası.
 - C-listesi: {T2 }
 - redo T2: B=10; C=15;
 - <ABORT T3> log dosyasına yaz.
- 9.satır ise;
 - Daha evvel bir <START CKPT(..)> yok. O zaman dosyanın başından başla.
 - C-listesi: {T1 }
 - redo T1: A=5;
 - <ABORT T2>; <ABORT T3> log dosyasına yaz.

Aktif denetim noktası (*nonquiescent checkpoint*)

UNDO-REDO'de AKTİF CKPT PROTOKOLÜ:

- $\langle \text{START CKPT } (T_1, \dots, T_k) \rangle$ log kaydını log dosyasına yaz. $T_1, \dots, T_k$, CKPT başladığı zaman aktif olan (commit olmamış olan) hareketlerdir.
- $\langle \text{START CKPT } (T_1, \dots, T_k) \rangle$ diske yazıldıktan sonra bu ana kadar **BÜTÜN kirli tamponları** diske yaz. **Bu arada yeni ortaya çıkan hareketlere mani olma.**
- $\langle \text{END CKPT} \rangle$ log kaydını diske yaz.

KURTARMA YÖNTEMİ:

Aynı undo-redo gibi. Fakat, Sondan başa doğru giderken;

- **Eğer $\langle \text{END CKPT} \rangle$ gördüysek;** $\langle \text{START CKPT } (T_1, \dots, T_k) \rangle$ log kaydındaki önceye
 - $T_1, \dots, T_k$ 'dan commit olmamış olanları UNDO yapmak için en eski $\langle \text{START } T_i \rangle$ log kaydına kadar gitmek gerekiyor.
 - $T_1, \dots, T_k$ 'dan commit olmuş olanlarda REDO yapmaya gerek yok. (Çünkü bütün kirli tamponlar diske yazıldı) Bundan sonra ileriye doğru giderken görünen $\langle T, X, v \rangle$ de T, C-listesindeyse redo yapılır. Değilse $\langle \text{ABORT } T \rangle$ yazılır.
- Eğer $\langle \text{END CKPT} \rangle$ görmeden, $\langle \text{START CKPT } (T_1, \dots, T_k) \rangle$ gördüysek; evvelki $\langle \text{END CKPT} \rangle$ ve ona ait olan $\langle \text{START CKPT } (S_1, \dots, S_m) \rangle$ log kaydına kadar gideriz. Bu sırada Commit olmayan hareketler undo yapılır. C-listesi oluşturulur. İleriye doğru giderken ise görünen $\langle T, X, v \rangle$ de T, C-listesindeyse redo yapılır. Değilse $\langle \text{ABORT } T \rangle$ yazılır.

Örnek: undo-redo'da CKPT

Sistem ilk çalıştığında log dosyasında görünen son satır:

1. <START T1>
2. <T1,A,4,5>
3. <START T2>
4. <COMMIT T1>
5. <T2,B,9,10>
6. **<START CKPT (T2)>**
7. <T2,C,14,15>
8. <START T3>
9. <T3,D,19,20>
10. **<END CKPT>**
11. <COMMIT T2>
12. <COMMIT T3>

- 9.satır ise;
 - Dosyanın başına kadar git.
 - REDO T1, UNDO T2; UNDO T3
 - Undo T3: D=19
 - Undo T2: C=14, B=9;
 - Redo T1: A=5

- 12.satır ise;
 - <START CKPT(..)>'e kadar git.
 - REDO T2 ve T3
 - Redo T2: ~~B=10;~~ C=15;
 - Redo T3: D=20
- 11.satır ise;
 - <START CKPT(..)>'e kadar git.
 - REDO T2; UNDO T3
 - Undo T3: D=19
 - Redo T2: ~~B=10;~~ C=15;
- 10.satır ise;
 - UNDO T2; UNDO T3
 - Undo T3: D=19
 - <START T2>ye kadar git.
 - Undo T2: C=14, B=9;

Örnek: *redo-only*

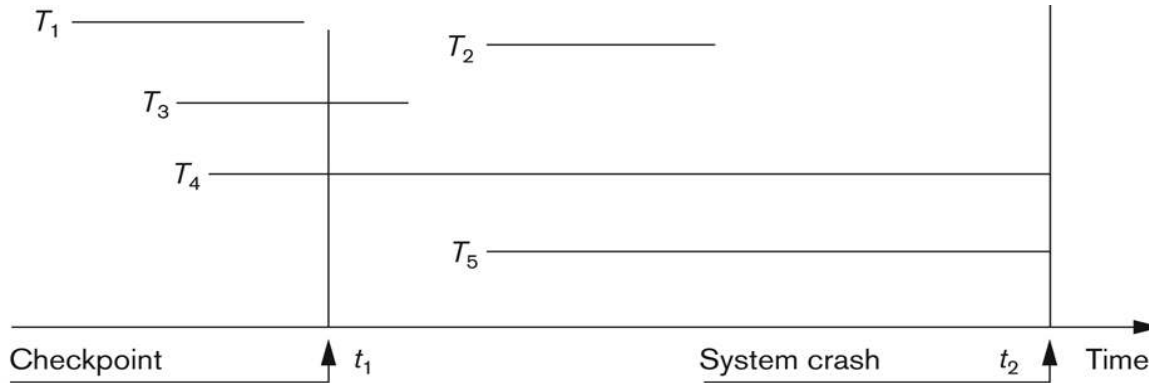


Figure 19.3
An example of
recovery in a multi-
user environment

■ Redo-only kurtarma

- Log kayıtlarında 2 tür hareket kaydı var: Commit edilmişler, aktif listesi
- Denetim noktasına kadar karşılaşılan Commit edilmişler –log dosyasına yazılma sırasında- redo yapılmalı, aktif listedeki hareketlere herhangi bir işlem yapılmamalı.

■ Örnekte,

- T1 için herhangi bir işlem yapılmaz.
- T4, T5 için herhangi bir işlem yapılmaz.
- T2 ve T3'ün işlemleri redo yapılmalı. (*Gerçekleme: Log dosyasını geriye doğru işlenebilir. İlk yapılan redo «redo listesi»ne eklenir. Log dosyasında geriye doğru ilerlerken ancak redo listesinde olmayan işlemler redo yapılabilir.)*

Kurtarma Yönetimi

- **KY:**

- KY-1: undo-redo, KY-2:undo-only, KY-3:redo-only
- Pasif, Aktif denetim noktası
- Seyrek Hatalardan Kurtarma
- Kurtalabilirlik Yönünden Plan Çeşitleri (*Recoverable, Cascadeless, Strict*)
- SimpleDB'de KY

Seyrek hatalardan kurtarma

- Seyrek hatalar: Disk hataları, fiziksel veya dışsal felaketler..
- ARŞİV: Daha güvenli başka bir saklama unitesi.
 - ARCHIVE_VT: VT'nin en son tutarlı hali.
 - ARCHIVE_LOG: VT'nin en son tutarlı halinden çökme zamanına en yakın zamanda arşivlenebilmiş olan LOG dosyası
- Arşivleme zaman alan bir işlem:
 - TAMARŞİV: bütün VT arşivlenir.
 - ARTIMLIARŞİV : en son ARŞİV'den itibaren yapılmış olan değişiklikler arşivlenir.
- AktifCKPT, ana hafızayı diske flush ederken hareketlerin yenilemeleri devam etti. Daha sonra log dosyası ile VT tutarlı bir noktaya taşındı.
 - Benzer şekilde Archive işlemi sırasında (DUMP), AktifCKPT sayesinde diskteki yenilemeler devam eder.
 - Disk hataları, **ARCHIVE_VT ve ARCHIVE_LOG dosyası sayesinde (undo-redo işlemleri ile)** alınabilecek en yakın geçmiş noktaya getirilebilir.

Örnek DUMP işlemi

- VT'da, arşive kopyalanan değerler «A=1,B=2,C=3,D=4» olsun.
- DUMP işlemi ve diskteki değişimlerin zamansal sırası:
A'yı kopyala; A=5; B'yı kopyala; C=6; C'yı kopyala; B=7; D'yı kopyala;
- Arşiv'deki ABCD değerleri: 1,2,6,4
- Bu sırada Arşive'e ulaşabilen Log dosyası (DUMP Log):

```
<START DUMP>  
<START CKPT (T1,T2)>  
<T1,A,1,5>  
<T2,C,3,6>  
<COMMIT T2>  
<T1,B,2,7>  
<END CKPT>  
Dump completes  
<END DUMP>
```

KURTARMA YÖNTEMİ:

ARŞİV'deki VT'nını diske yükle. (En son arşiv ve varsa artımlı arşiv bilgilerini kullanarak)

DUMP Log ile diskteki VT'ını tutarlı bir noktaya getir.

Örnek'te T1 commit olmadığı için yaptığı değişiklikler UNDO olur, T2 commit olduğu için REDO olur.

Kurtarma Yönetimi

- **KY:**
 - KY-1: undo-redo, KY-2:undo-only, KY-3:redo-only
 - Sessiz, sesli denetim noktası
 - Seyrek Hatalardan Kurtarma
 - Kurtalabilirlik Yönünden Plan Çeşitleri (*Recoverable, Cascadeless, Strict*)
 - SimpleDB'de KY

Kurtalabilirlik Yönünden Plan Çeşitleri

- Kurtarılamayan planlar : commit olan bir hareket rollback yapılması gerekiyorsa plan kurtarılmazdır.
- Kurtarılabilir Planlar (*recoverable schedules*) : Bir planda, okuma yaptığı başka hareketlerden(T') evvel commit olan bir hareket(T) yoksa plan kurtarılabilir. (T hareketinin T' hareketinden okuma yapması T' hareketinin yazdığı değeri T hareketinin okuması demek: $W'(x) R(X)$ gibi)

Sa: $r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); c_2; w_1(Y); c_1; \rightarrow$ kurtarılabilir.

Sc: $r_1(X); \mathbf{w_1(X)}; \mathbf{r_2(X)}; r_1(Y); w_2(X); \mathbf{c_2}; \mathbf{a_1}; \rightarrow$ kurtarılamaz.

Sd: $r_1(X); \mathbf{w_1(X)}; \mathbf{r_2(X)}; r_1(Y); w_2(X); w_1(Y); \mathbf{c_1}; \mathbf{c_2}; \rightarrow$ kurtarılabilir.

Se: $r_1(X); \mathbf{w_1(X)}; \mathbf{r_2(X)}; r_1(Y); w_2(X); w_1(Y); \mathbf{a_1}; \mathbf{a_2}; \rightarrow$ kurtarılabilir.

Kurtalabilirlik Yönünden Plan Çeşitleri

- Zincirsiz Kurtarılabilir Planlar (cascadeless schedules): Kurtarılabilir bir planda bir hareketin kurtarılması(geri sarılması) başka hareketlerin geri sarılmasına sebep olmuyorsa bu planlar «zincirsiz kurtarılabilir» planlardır.

Sd: $r_1(X)$; $\mathbf{w}_1(\mathbf{X})$; $\mathbf{r}_2(\mathbf{X})$; $r_1(Y)$; $w_2(X)$; $w_1(Y)$; $\mathbf{c}_1$; $\mathbf{c}_2$; $\rightarrow$ kurtarılabilir.

Se: $r_1(X)$; $\mathbf{w}_1(\mathbf{X})$; $\mathbf{r}_2(\mathbf{X})$; $r_1(Y)$; $w_2(X)$; $w_1(Y)$; $\mathbf{a}_1$; $\mathbf{a}_2$; $\rightarrow$ kurtarılabilir.

Sd: $r_1(X)$; $\mathbf{w}_1(\mathbf{X})$; $r_1(Y)$; $w_1(Y)$; $\mathbf{c}_1$; $\mathbf{r}_2(\mathbf{X})$; $w_2(X)$; $\mathbf{c}_2$; $\rightarrow$ zincirsiz kurtarılabilir.

Se: $r_1(X)$; $\mathbf{w}_1(\mathbf{X})$; $r_1(Y)$; $w_1(Y)$; $\mathbf{a}_1$; $\mathbf{r}_2(\mathbf{X})$; $w_2(X)$; $\mathbf{c}_2$; $\rightarrow$ zincirsiz kurtarılabilir.

- Katı Planlar (strict schedules) : Bir planda, T hareketinin değiştirdiği bir değeri başka bir T' hareketi, T hareketi commit olmadan değiştirmiyorsa plan STRICT planıdır.

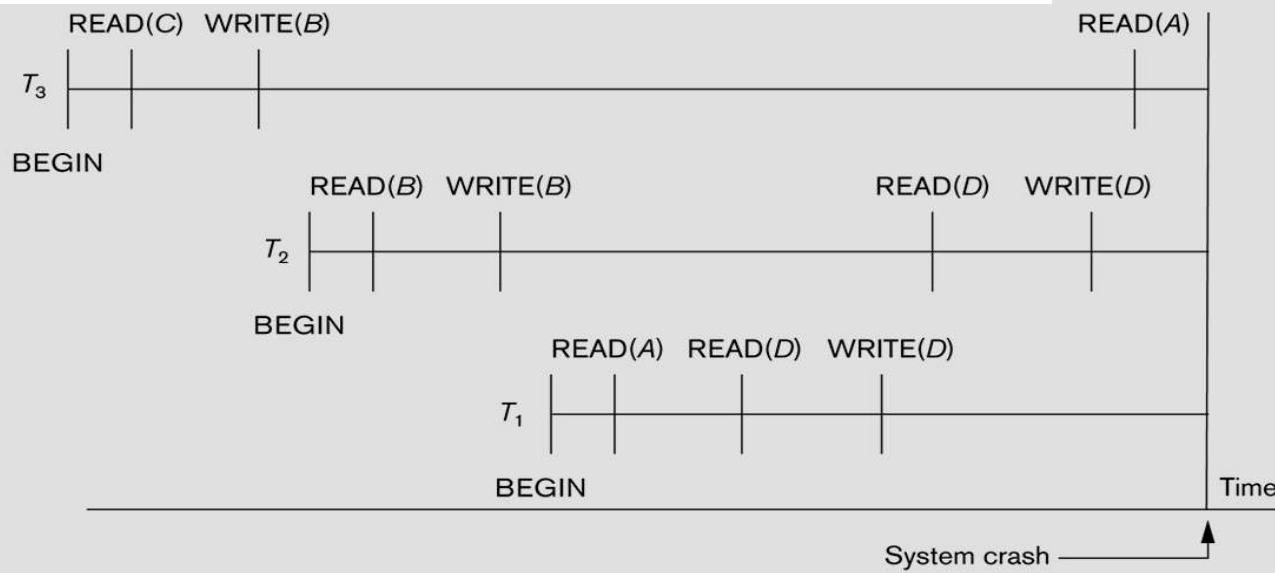
Sf : $w_1(X, 5)$; $\mathbf{w}_2(\mathbf{X}, 8)$; $\mathbf{a}_1$;

Örnek: (*undo-only, cascade rollback*)

T_1	T_2	T_3
read_item(A)	read_item(B)	read_item(C)
read_item(D)	write_item(B)	write_item(B)
write_item(D)	read_item(D)	read_item(A)
	write_item(D)	write_item(A)

	A	B	C	D
	30	15	40	20
[start_transaction, T_3]				
[read_item, T_3 , C]				
* [write_item, T_3 , B, 15, 12]		12		
[start_transaction, T_2]				
[read_item, T_2 , B]				
** [write_item, T_2 , B, 12, 18]		18		
[start_transaction, T_1]				
[read_item, T_1 , A]				
[read_item, T_1 , D]				
[write_item, T_1 , D, 20, 25]				25
[read_item, T_2 , D]				
** [write_item, T_2 , D, 25, 26]				26
[read_item, T_3 , A]				

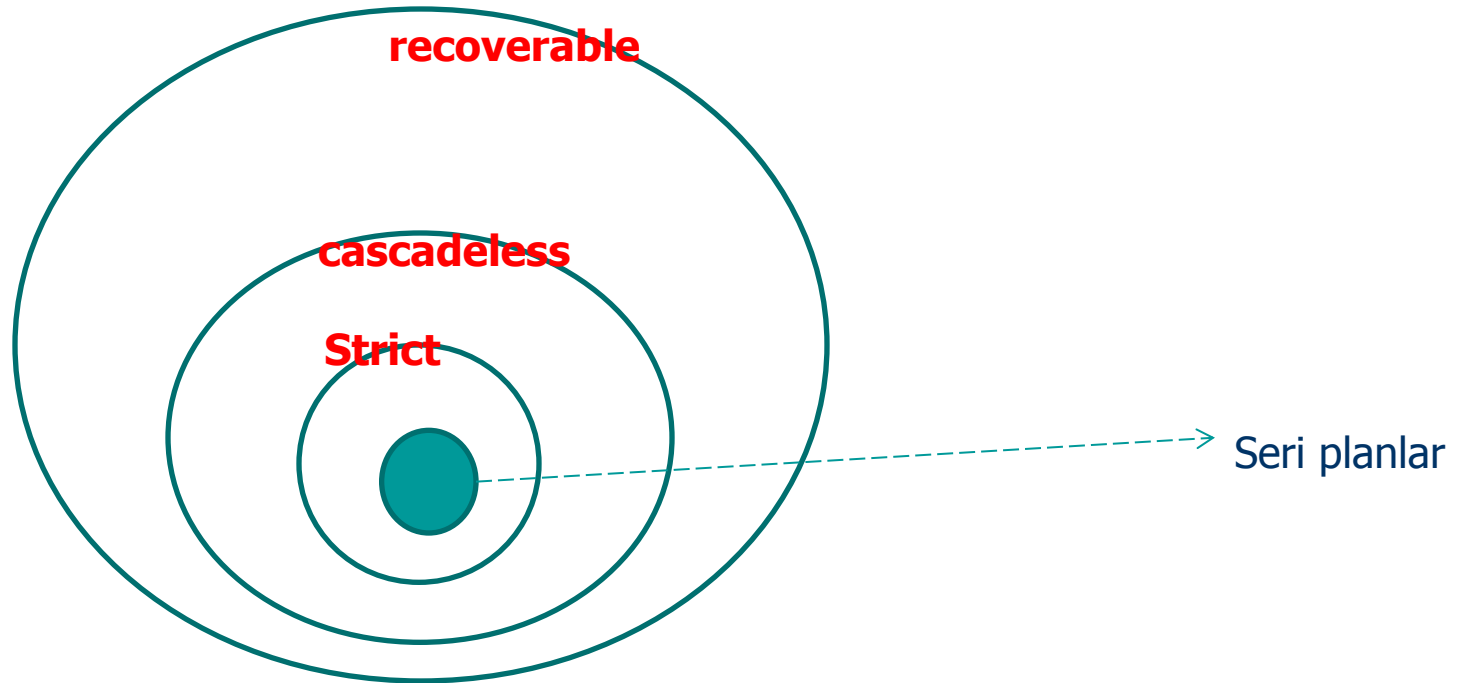
← System crash



- T_3 rollback → T_2 cascaded rollback.

Log dosyasında read kayıtlarının tutulmasının tek amacı «cascade rollback» durumunu tespit etmek içindir. Recovery algoritmaının çoğu **cascadeless olduğundan (ileride tanımlanıyor)** dolayı çoğuzaman «read» log kayıtları olmaz.

Kurtalabilirlik Yönünden Plan Çeşitleri



Kurtarma Yönetimi

- **KY:**
 - KY-1: undo-redo, KY-2:undo-only, KY-3:redo-only
 - Pasif, Aktif denetim noktası
 - Seyrek Hatalardan Kurtarma
 - Kurtalabilirlik Yönünden Plan Çeşitleri (Recoverable, Cascadeless, Strict)
 - SimpleDB'de KY

SimpleDB Kurtarma Yönetimi

- *simplifiedb.tx.recovery* paketi içindeki ***RecoveryManager*** sınıfı
- Her bir tx kendisi için 1 adet kurtarma yönetimi kullanır.
- SimpleDB'de,
 - undo-only kurtarma algoritmasını
 - değer-taneselliği ile gerçekleştirilmiştir.
- SimpleDB KY:
 - Log kayıtlarının gerçekleştirilmesi
 - Log dosyasındaki kayıtların yeniden okunması
 - geri sarma (*Rollback*) ve kurtarma (*recovery*)

KY-1:Log kayıtlarının gerçekleştirilmesi

- Bu log tiplerinin herbiri 1 sınıf ile gerçekleştirilmiş.

log kayıt tipleri	Gerçekndiği sınıf
CHECKPOINT	CheckpointRecord
START	StartRecord
COMMIT	CommitRecord
ROLLBACK	RollbackRecord
SETINT	SetintRecord
SETSTRING	SetstringRecord

- Her log kayıt sınıfı LogRecord arayüzünü implement ediyor.

```
public interface LogRecord {  
    static final int CHECKPOINT = 0, START = 1,  
                    COMMIT = 2, ROLLBACK = 3,  
                    SETINT = 4, SETSTRING = 5;  
  
    static final LogMgr logMgr = SimpleDB.logMgr();  
  
    int writeToLog(); Kaydı log dosyasına yazar LSN'yi dondurur  
    int op();  
    int txNumber();  
    void undo(int txnum); SETINT ve SETSTRING için  
}
```

Örnek: *SetStringRecord* sınıfı

```
public class SetStringRecord implements LogRecord {
    private int mytxnum, offset;
    private String val;
    private Block blk;

    public SetStringRecord(int txnum, Block blk,
                           int offset, String val) {
        this.txnum = txnum;
        this.blk = blk;
        this.offset = offset;
        this.val = val;
    }
}
```

- 2 constructor var:
 - Birincisi log dosyasına yazmadan hemen önce kullanılır.
 - Diğer rollback ve recovery tarafından kullanılır. Bunlar önce LogManager'dan BasicLogRecord kaydını alıp SetStringRecord ile karşılık gelen nesne üretilir.

```
public SetStringRecord(BasicLogRecord rec) {
    mytxnum = rec.nextInt();
    String filename = rec.nextString();
    int blknum = rec.nextInt();
    blk = new Block(filename, blknum);
    offset = rec.nextInt();
    val = rec.nextString();
}

public int writeToLog() {
    Object[] rec = new Object[] {SETSTRING, txnum,
                                  blk.fileName(), blk.number(), offset, val};
    return logMgr.append(rec);
}

public int op() {
    return SETSTRING;
}

public int txNumber() {
    return mytxnum;
}

public void undo(int txnum) {
    BufferMgr buffMgr = SimpleDB.bufferMgr();
    Buffer buff = buffMgr.pin(blk);
    buff.setString(offset, val, txnum, -1);
    buffMgr.unpin(buff);
}

public String toString() {
    return "<SETSTRING " + mytxnum + " " + blk
        + " " + offset + " " + val + ">";
}
```

KY-2: Log dosyasındaki kayıtların yeniden okunması (*iterate*)

```
class LogRecordIterator implements Iterator<LogRecord> {  
    private Iterator<BasicLogRecord> iter =  
        SimpleDB.logMgr().iterator();  
  
    public boolean hasNext() {  
        return iter.hasNext();  
    }  
  
    public LogRecord next() {  
        BasicLogRecord rec = iter.next();  
        int op = rec.nextInt();  
        switch (op) {  
            case CHECKPOINT:  
                return new CheckpointRecord(rec);  
            case START:  
                return new StartRecord(rec);  
            case COMMIT:  
                return new CommitRecord(rec);  
            case ROLLBACK:  
                return new RollbackRecord(rec);  
            case SETINT:  
                return new SetIntRecord(rec);  
            case SETSTRING:  
                return new SetStringRecord(rec);  
            default:  
                return null;  
        }  
    }  
}
```

■ Örnek:

```
Iterator<LogRecord> iter=new  
    LogRecordIterator();  
while(iter.hasNext()){  
    LogRecord rec=iter.next();  
    System.out.println(rec.toString());  
}
```


KY-3: geri sarma ve kurtarma

```
public class RecoveryMgr {  
    private int txnum;  
  
    public RecoveryMgr(int txnum) {  
        this.txnum = txnum;  
        LogRecord rec = new StartRecord(txnum);  
        rec.writeToLog();  
    }
```

Undo-only gerçekleşiyor..

```
    public void commit() {  
        SimpleDB.bufferMgr().flushAll(txnum);  
        LogRecord rec = new CommitRecord(txnum);  
        int lsn = rec.writeToLog();  
        SimpleDB.logMgr().flush(lsn);  
    }  
  
    public void rollback() {  
        doRollback();  
        SimpleDB.bufferMgr().flushAll(txnum);  
        LogRecord rec = new RollbackRecord(txnum);  
        int lsn = rec.writeToLog();  
        SimpleDB.logMgr().flush(lsn);  
    }
```

```
    public void recover() {  
        doRecover();  
        SimpleDB.bufferMgr().flushAll(txnum);  
        LogRecord rec = new CheckpointRecord();  
        int lsn = rec.writeToLog();  
        SimpleDB.logMgr().flush(lsn);  
    }  
  
    public int setInt(Buffer buff, int offset,  
                      int newval) {  
        int oldval = buff.getInt(offset);  
        Block blk = buff.block();  
        if (isTemporaryBlock(blk))  
            return -1;  
        else {  
            LogRecord rec =  
                new SetIntRecord(txnum, blk, offset, oldval);  
            return rec.writeToLog();  
        }  
    }  
  
    public int setString(Buffer buff, int offset,  
                          String newval) {  
        String oldval = buff.getString(offset);  
        Block blk = buff.block();  
        if (isTemporaryBlock(blk))  
            return -1;  
        else {  
            LogRecord rec =  
                new SetStringRecord(txnum, blk, offset, oldval);  
            return rec.writeToLog();  
        }  
    }  
  
    private void doRollback() {  
        Iterator<LogRecord> iter = new LogRecordIterator();  
        while (iter.hasNext()) {  
            LogRecord rec = iter.next();  
            if (rec.txNumber() == txnum) {  
                if (rec.op() == START)  
                    return;  
                rec.undo(txnum);  
            }  
        }  
    }
```


KY-3: geri sarma ve kurtarma

Undo-only gerçekleşiyor..rollback tespit edilmiyor..

```
private void doRecover() {
    Collection<Integer> committedTxns =
        new ArrayList<Integer>();
    Iterator<LogRecord> iter = new LogRecordIterator();
    while (iter.hasNext()) {
        LogRecord rec = iter.next();
        if (rec.op() == CHECKPOINT)
            return;
        if (rec.op() == COMMIT)
            committedTxns.add(rec.txNumber());
        else if (!committedTxns.contains(rec.txNumber()))
            rec.undo(txnum);
    }
}

private boolean isTemporaryBlock(Block blk) {
    return blk.fileName().startsWith("temp");
}
```

Eşzamanlılık Yönetimi

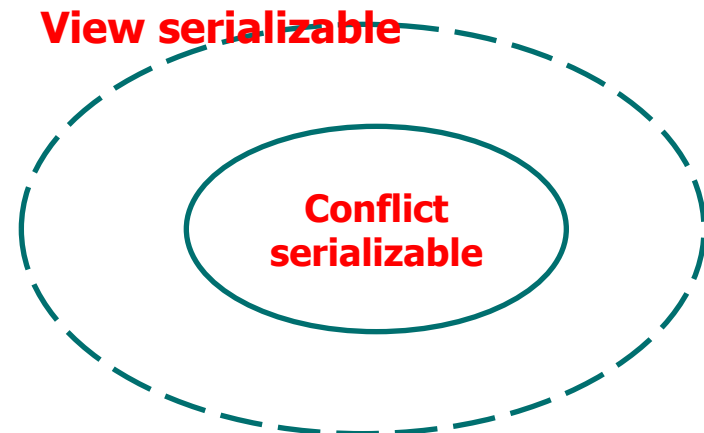
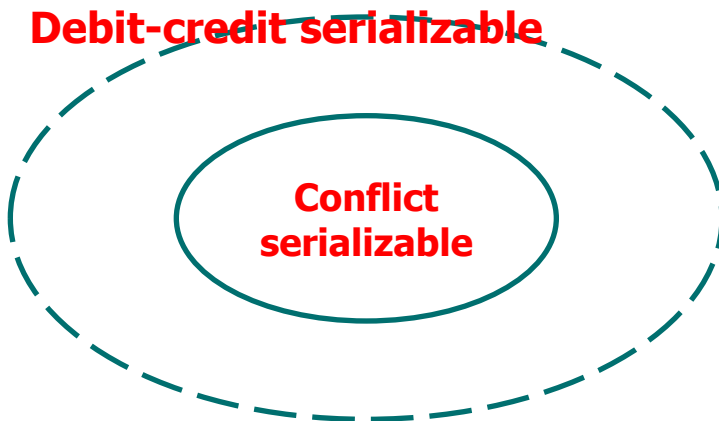
- **Serilenebilirlik: (C-S, V-S, DC-S)**
- Temel Kilit Protokolü ve 2PL ve versiyonları
- Ölüklilit ve Açlık
- Kilitler ile Yalıtım seviyelerinin sağlanması
- Diğer Kilit Protokolleri (Kilit Yükseltme, Update Kilit, Increment kilit, MGL kilit)
- Phantom
- SimpleDB'de eşzamanlılık kontrolü ve Hareket Yönetimi
- Diğer Eşzamanlılık Protokolleri
 - MV-lock
 - TimeStamp
 - Optimistic methods

Eşzamanlılık Yönetimi (EY)

- Amaç: Eşzamanlı tx'ların herbirinin **doğru olarak** sonlanması.
- Bazı Tanımlar:
 - **Veri tabanı eylemi (db action)**: bir bloğun diskten okunması-yazılması
 - **Tarihçe**: bir tx'nın vt dosyaları üzerindeki erişim dizisi. Örneğin:
 - tx: R(student.tbl,0); R(student.tbl,0); W(student.tbl,0);
 - **Plan**: vt üzerinde işlem yapan bütün tx'ların (*öngörülemmez halde*) içiçe girmiş tarihçeleri. Örneğin: T1: W(b1); W(b2) T2: W(b1);W(b2)
Plan1 : W₁(b1); W₂ (b1); W₂ (b2); W₁ (b2)
 - Her plan doğru bir vt durumu ile sonlanmayabilir.
 - **Seri Plan**: tx tarihçelerinin içiçe girmemiş (birbirinden izole, sırayla) olduğu plan.
 - Örneğin; önceki örnekteki plan seri değil.
 - Birden çok seri plan olabilir (T1;T2 veya T2;T1). Bunlar birbirinden farklı vt durumu ile sonlanabilir; ve **hepsi doğrudur**. Çünkü tx'lar tamamiyle izole!..
 - **Serilenebilir Plan**: Herhangi bir seri plan ile aynı vt durumunu netice veren plan. Demek ki; serilenebilir plan doğrudur. Yukarıdaki Plan1 serilenebilir bir plan değil; demek ki yanlış. Aşağıdaki Plan2=T1;T2 ?
 - Plan2: W₁(b1); W₂ (b1) ; W₁ (b2); W₂ (b2)

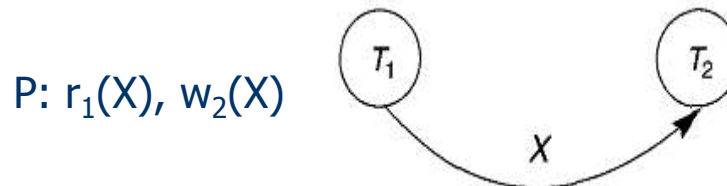
Serilenebilirlik

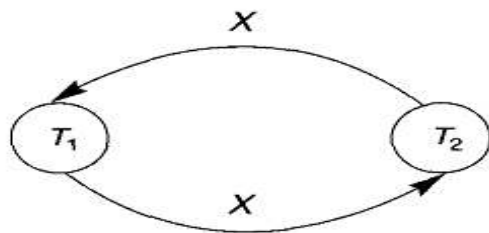
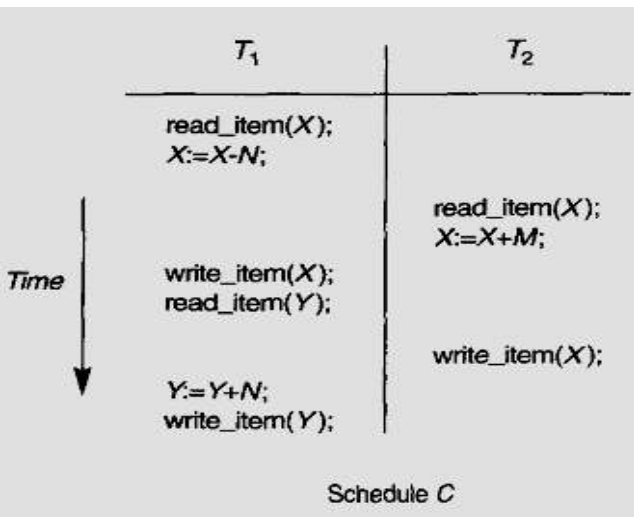
- **Çelişki:** Farklı sırada çalışması, sonucu değiştiren **farklı hareketlerdeki** eylem ikililerinin eşzamanlı çalışması. Plandaki **çelişkiler** planın serilenebilir olmasını etkiler.
- **2 operasyonun Çelişmesi için 3 şart:**
 - Farklı hareketlerde olmaları
 - En az birinin W olması
 - Aynı veri parçası üzerinde işlem yapması
- T1: R(b1); W(b2) T2: W(b1); W(b2)
 $W_1(b2)$ ve $W_2(b2)$ yazma-yazma çelişkisi
 $R_1(b1)$ ve $W_2(b1)$ okuma-yazma çelişkisi



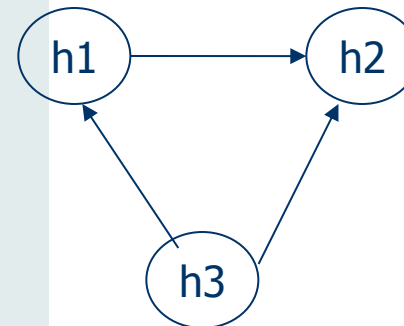
«Çelişki esaslı Serilenebilirlik» *conflict serializable (C-serializable)*

- Gerçek VT'larında en çok kullanılan yöntem.
- İki planın «çelişki esaslı denk» (*conflict equal*) olması için şart; BÜTÜN çelişen ikililerin işlenmesi sırası aynı olması gerek.
- Seri olmayan bir plan, seri bir plana çelişki esaslı denk ise bu plan «çelişki esaslı serilenebilir» (*conflict serializable*) bir plandır.
- «Çelişki esaslı Serilenebilirlik» testi için 2 yol:
 - Bir P, planı, ÇELİŞMEYEN işlemlerinin sırası değiştirilerek seri plana dönüştürülebilirse, P'ye «çelişki esaslı serilenebilir» (*conflict serializable*) denir.
 - Precedence graph (öncelik testi): «precedence graph»: Öncelik grafi çevrim içermiyorsa, plan «çelişki esaslı serilenebilir (conflict serializable) bir plandır" denilir.
 - Her bir hareket için bir düğüm oluşturulur.
 - Her bir çelişen operasyon için iki hareekt arasına ok çizilir.
 - Okun yönü çelişen operasyonlardan ilkinin çalıştırılan hareketten diğerine doğru.

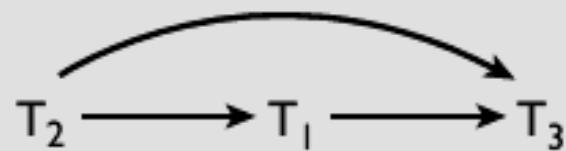




H1	H2	H3
R(A) W(A)		R(B) R(C)
	R(C)	W(B) W(C)
R(B) W(B)	R(B) W(B) R(A) W(A)	



$r_1[x] \ r_2[x] \ w_1[x] \ r_3[x] \ w_2[y] \ c_2 \ w_1[y] \ c_1 \ w_3[x] \ c_3$



«Görüntü esaslı Serilenebilirlik» *view serializable (V-serializable)*

S1: w1(Y); w1(X); w2(Y); w2(X); w3(X);

S2: w1(Y); w2(Y); w2(X); w1(X); w3(X);

- S2 doğru mu (serilenebilir mi)? Çelişki esaslı serilenebilir mi?
- Daha esnek bir serilenebilir plan: View Serializable
- **S ve S' planlarının «görüntü esaslı denk» (*view equal*) olması için şart;**
 1. S ve S' deki hareketler aynı hareketlerdir.
 2. S de X değerini okuyan i hareketi, $R_i(X)$; evvelinde X değerini en son değiştiren j hareketinden sonra işlendi ise; bu durum (T_i 'nin T_j 'yi görmesi) S' de de böyle olmalıdır. (*Burası C-Serilenebilirlik ile aynı*)
 3. S de Y değerini en son değiştiren hareket T_k ise; S' de de bu durum böyle olmalıdır.
- **BLIND W:** Değiştirilecek değeri VT'dan okumadan, yeni değeri VT'na yazmak.
- Plandaki hareketlerde «Blind W» yoksa «view» = «conflict»
- S: r1(x);w2(X);w1(X);w3(X);c1;c2;c3. → C-S değil fakat V-S →
= $\langle T_1, T_2, T_3 \rangle$

«Borç-alacak Serilenebilirlik» *debit-credit serializable (dc-serializable)*

- Daha esnek bir serilenebilir plan.
- Hareket işlemleri bütünlüğünü ($r(X)$; do_s/t_with_x ; $w(X)$) muhafaza etmeli.
 - do_s/t_with_x : + veya -

S:

T1

R(A)
A=A+100
W(A)

T2

R(A)
A=A+200
W(A)
R(B)
B=B+200
W(B)

R(B)
B=B+100
W(B)

$$\langle T1, T2 \rangle = \langle T2, T1 \rangle = S$$

SONUÇ:

Eşzamanlılık kontrolünün hareketin işlem detaylarını bilmesi kolay olmadığı için gerçekleşmesi zor olan eşzamanlılık seviyesi.

Gerçek hayat

- Hareketler ortaya çıkarken planların Serilenebilirliğini test etmek ZOR. (*NP-hard problem*).
- Çoğu sistemde serilenebilirlik 2PL-kilit protokolü ile conflict-serializable seviyesinde sağlanıyor.

Eşzamanlılık Yönetimi

- Serilenebilirlik: (C-S, V-S, DC-S)
- **Temel Kilit Protokolü ve 2PL ve versiyonları**
- **Ölökilit ve Açlık**
- Kilitler ile Yalıtım seviyelerinin sağlanması
- Diğer Kilit Protokolleri (Kilit Yükseltme, Update Kilit, Increment kilit, MGL kilit)
- Phantom
- SimpleDB'de eşzamanlılık kontrolü ve Hareket Yönetimi
- Diğer Eşzamanlılık Protokolleri
 - MV-lock
 - TimeStamp
 - Optimistic methods

Kilitleme ve 2PL:

- EY modülü; planların serilenebilir (yani doğru) olmasından sorumludur. Bunu kilitleme ile sağlar.

- Blok kilitleme:

- Paylaşımlı kilit (slock)
- Dışlayıcı kilit (xlock)

- Kilit uyumluluk matrisi:

«B» Veri parçası
üzerinde mevcut kilit

	«B» Veri parçası için istenen kilit	
	S	X
S	Yes	No
X	No	No

- ÖRNEK: $W_1(b1)$; $W_2(b1)$; $W_2(b2)$; $W_1(b2)$

XL1(b1); $W1(b1)$; **UL1(b1)**; **XL2(b1)**; $W2(b1)$; $UL2(b1)$; **XL2(b2)**; $W2(b2)$;
UL2(b2); **XL1(b2)**; $W1(b2)$; **UL1(b2)**

- ÖRNEK: $R_1(b1)$; $W_2(b1)$; $W_1(b2)$; $W_2(b2)$

SL1(b1); $R1(b1)$; **UL1(b1)**; **XL2(b1)**; $W2(b1)$; **UL2(b1)**; **XL1(b2)**;
 $W_1(b2)$; **UL1(b2)**; **XL2(b2)**; $W2(b2)$; **UL2(b2)**

- Kilit ile kaynaklara erişim serilenebilirlik (doğruluk) için yeterli değil. Kilidin bırakılma zamanı da kontrollü olması gerekiyor. Bunu sağlayan protokol 2PL'dir.

Kilidi erken bırakmanın sorunları

- Tx bir kiliti (daha sonra o bloğu kullanmasa bile) erken bırakırsa;

- Serilenebilirlik problemi:

T1:SL(x),R(x); UL(x); SL(y);R(y);.....



T2: W(x);W(y);commit

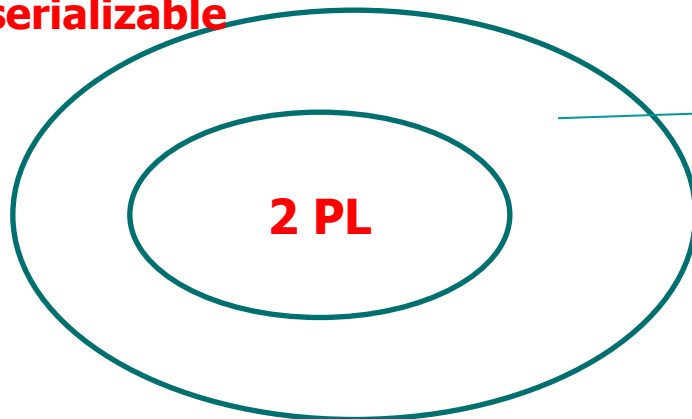
- Commit edilmemiş veriyi okuma problemi: (*zincirli rollback veya kurtarılamaz planlar*)

.....W₁(b); UL₁(b); SL₂(b);R₂(b);c2..... rollback tx1

2-phase locking (*2-aşamalı kilit protokolü*)

- "Bir hareket, gerekli bütün kilitlerini almadan önce herhangi bir kilidi bırakamaz» veya «bir kilit bırakıldıktan sonra bir daha herhangi bir başka kilit alınamaz» esasına göre çalışan hareketler **2PL'dir** → "SERİLENEBİLİR" dir.
- ANCAK,
 - «SERİLENEBİLİR → 2PL'dir». DIYEMEYİZ.
 - Veya «2PL değil» → «SERİLENEBİLİR DEĞİL» DIYEMEYİZ.
 - ÇÜNKÜ;
 - V-S ve DC-S'yi hatırlayalım.
 - Diğer doğru olabilecek durumlar. (verilere yapılan işlemlere göre plan C-S, V-S veya DC-S olmasa da doğru olabilir. *Bunlar konu kapsamı dışında.*

serializable



«w1(x) w3(x) w2(y) w1(y)» için 2PL plan yazamayız. Ancak bu plan serilenebilir.

2PL → «serilenebilir»

- $T_1, T_2, \dots, T_k$: 2-phase-lock protokolüne uyan hareketler olsun. Bu durumda öncelik grafinin çevrim içermesinin mümkün olamayacağını gösterelim.
- $T_1 \dashrightarrow T_2$: çelişen 2 operasyon var. T_1 , kilit bırakmadan T_2 kilit alamaz.
- $T_1 \dashrightarrow T_2 \dashrightarrow T_3$: T_1 , kilit bırakmadan T_3 kilit alamaz.
- $T_1 \dashrightarrow T_2 \dashrightarrow \dots \dashrightarrow T_k$
- $T_1 \dashrightarrow T_2 \dashrightarrow \dots \dashrightarrow T_k \dashrightarrow T_1$: T_1 , kilidi bırakmadan T_1 kilidi alamaz. Bu ise 2-phase lock'a aykırı. O zaman çevrim içeren bir öncelik grafi oluşması mümkün değil.

2PL uyumlu Kilit istekleri senaryosu:

Three threads wish to lock blocks b_1 and b_2 as follows:

Thread A: sLock(b_1); sLock(b_2); unlock(b_1); unlock(b_2);

Thread B: xLock(b_2); sLock(b_1); unlock(b_1); unlock(b_2);

Thread C: xLock(b_1); sLock(b_2); unlock(b_1); unlock(b_2);

Suppose that the threads happen to execute so that they are interrupted after each statement. Then:

1. Thread A obtains an slock on b_1 .
2. Thread B obtains an xlock on b_2 .
3. Thread C cannot get an xlock on b_1 , because someone else has a lock on it. Thus thread C waits.
4. Thread A cannot get an slock on b_2 , because someone else has an xlock on it. Thus thread A also waits.
5. Thread B can continue. It obtains an slock on block b_1 , because nobody currently has an xlock on it. (It doesn't matter that thread C is waiting for an xlock on that block.)
6. Thread B unlocks block b_1 , but this does not help either waiting thread.
7. Thread B unlocks block b_2 .
8. Thread A can now continue, and obtains its slock on b_2 .
9. Thread A unlocks block b_1 .
10. Thread C finally is able to obtain its xlock on b_1 .
11. Threads A and C can continue in any order until they complete.

Örnek: 2-phase locking

P1 (2PL değil VE «serilenebilir değil; çünkü CYCLE var.»)	
H1	H2
SL(b) R(b) UL(b)	SL(a) R(a) UL(a) XL(b) R(b) b=b+2 W(b) UL(b)
XL(a) R(a) a=a+b W(a) UL(a)	

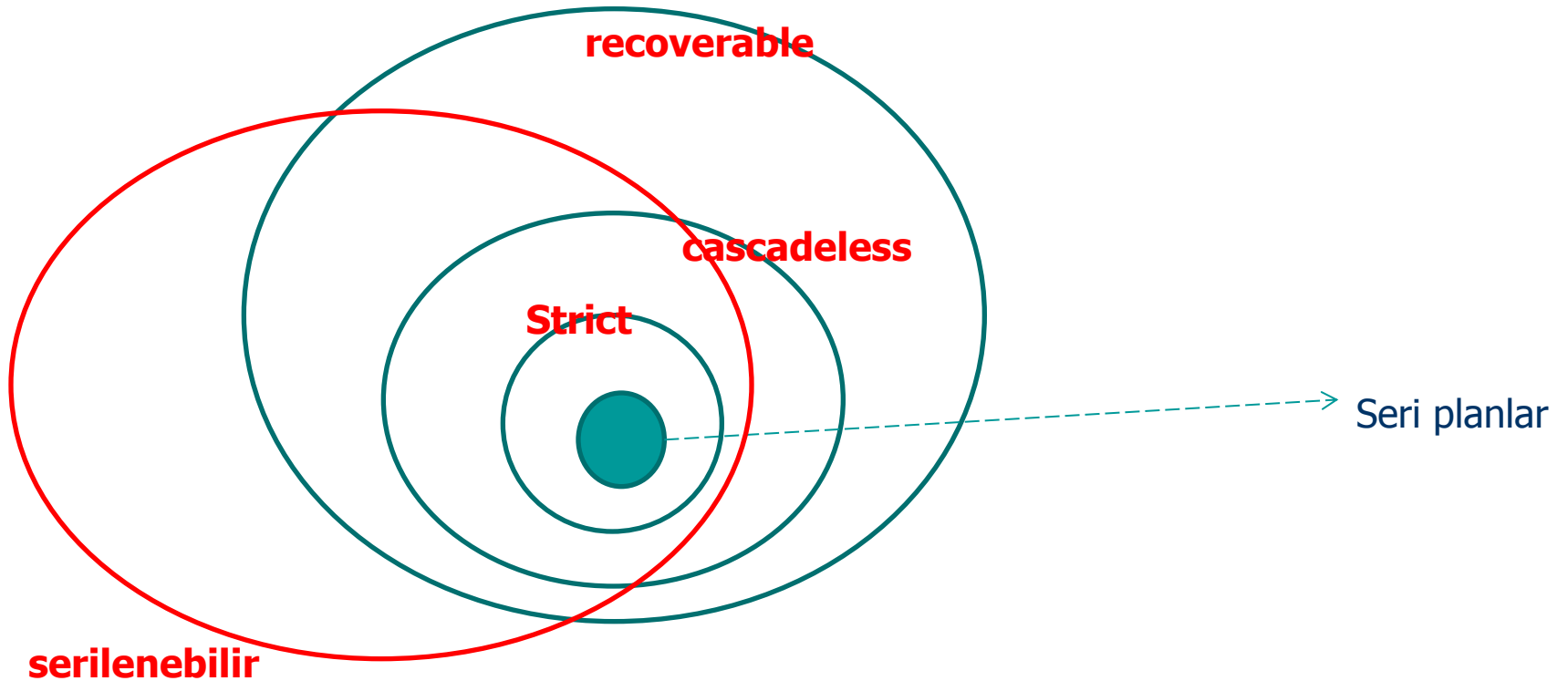
P2 (2PL → serilenebilir)	
H1	H2
SL(b)	
R(b)	
XL(a)	
UL(b)	
	XL(b)
	R(b)
R(a)	
a=a+b	
W(a)	
UL(a)	
	SL(a)
	R(a)
	b=b+a
	W(b)
	UL(b)
	UL(a)

2PL Kilit Protokolü Versiyonları:

basic, conservative, strict ve rigorous 2PL protokolleri

- Basic: standart 2 evreli olan 2PL protokolü.
 - Cascade rollback problemi var.
 - Deadlock olabilir.
- Conservative-2PL: Bütün kilitler hareket başlamadan evvel alınır.
 - Deadlock olmuyor.
 - Hareket başlaması ile kilit bırakmalar başlıyor.
 - pratikte kullanımı zor çünkü hareket başlamadan r ve w setin belli olması çok kolay değil.
- Strict 2PL: W kilitin bırakılması hareket commit olmasına kadar ertelenir.
 - "strict schedule" tipte kurtarılabilir planları netice verir.
 - Deadlock olabilir.
 - Cascade rollback problemi yok.
- Rigorous 2PL: Bütün kilitlerin (W ve R) bırakılması hareket commit oluncaya kadar ertelenir.
 - **Gerçeklenmesi strict2PL'ye göre daha da kolaydır.**
 - Hareket commit oluncaya kadar kilit toplama devam edebilir.
 - Deadlock olabilir.
 - Cascade rollback problemi yok.

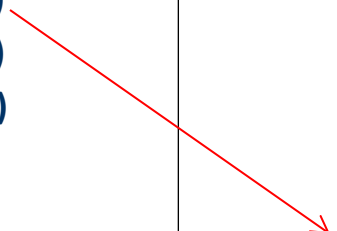
Kurtalabilirlik/Serilenebilir Yönünden Plan Çeşitleri



Basic 2PL → cascade rollback problemi

P2 (2PL, serilenebilir)

H1	H2
XL(b) R(b) W(b) XL(a) UL(b)	XL(b) R(b)
R(a) a=a+b W(a) UL(a)	SL(a) R(a) b=b+a W(b) UL(b) UL(a)



- H1 abort olursa, H2 de zincirli bir şekilde abort olmalı.
- 2PL, zincirsiz rollback'ı garanti etmiyor. Ancak Strict 2PL ve Rigorous 2PL'de zincirli rollback yok.

Strict 2PL → deadlock

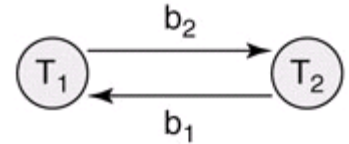
P2 (serilenebilir)	
H1	H2
XL(a)	SL(b) R(b) SL(a)
R(a) W(a) XL(b)	

- zincirsiz geri dönüşüm yok.
- kilit bırakmamak → deadlock ihtimali yüksek.
- Strict 2PL, rigorous 2PL → deadlock ihtimali artıyor.
- "2-phase lock Kilit Protokolleri", Serilenebilir Planı garanti ediyor ANCAK,
 - Ölükilitsiz çalışmayı garanti etmiyor. (sadece Conservative-2PL garanti eder, ancak o da pratikte çok mümkün değil)
- 2PL, BÜTÜN SERİLENEBİLİR PLANLARI GERÇEKLER Mİ?
 - HAYIR.

Ölökilit (*deadlock*)

- Birden çok hareketin karşılıklı birbirini bloke etmesi durumu. Tipik olarak %1 karşılaşılan bir durum.

- T1: W(b1); W(b2) T2: W(b2);W(b1)
- T1: xlock(b1); pin(b4) T2: pin(b2); pin(b3);xlock(b1)
- Ölökilit'in tespit edilmesi EY modulünün görevi.



- **Kesin ölökülüt tespiti (*deadlock detection*)**

- "waits-for" graf: Grafi saklamak, organize etmek çevrimleri tespit etmek zorlaşabiliyor.
- Kısa ve hafifi hareketlerde tercih edilir.

- **Yaklaşık ölökilit tespiti (*deadlock prevention*): $TS(T1) < TS(T2)$, T1 daha yaşlı ise ve T1, T2 tarafından tutulan bir kilidi istiyorsa:**

- "wait-die" : T1, T2'nin kilidi bırakmasını bekler. Diğer durum (kilit isteyen genç ise) ölür.
- «Wound-wait» : T2'yi abort et. Diğer durum (kilit isteyen genç ise) bekler.
- Zaman ayarlı yöntemler: T1 belirli bir süre ilerliyemiyorsa, deadlock olduğu varsayılır ve abort olur.

Açlık (*starvation*)

- STARVATION (açlık): kaynak tahsisinde (kilit, tampon gibi) bazı hareketlerin «talihsiz» bir şekilde kaynak alamamasıdır.
- Çözümler:
 - Adil kaynak kullanımı (FIFO veya CLOCK gibi)
 - «Öncelik» (priority) düzenlemesi varsa; düşük öncelikli hareketlerin öncelikleri zamanla artırılmalı.
 - Wait-die veya wound-wait'de ölen genç hareketin yeniden başlarken evvelki TS değeri ile sisteme dahil olması. (*ta ki yaşlanabilsin*)

Örnek

- $T_1: W(b_1); R(b_2); W(b_1); R(b_3); W(b_3); R(b_4); W(b_2);$
- $T_2: R(b_2); R(b_3); R(b_1); W(b_3); R(b_4); W(b_4);$

Yukarıdaki hareketler ile 2PL ile «serilenebilir» PLAN/ar yazınız?

- PLAN1: $\langle T_2, T_1 \rangle$

$R_2(b_2); R_2(b_3); \cancel{R_2(b_1)}; \cancel{W_1(b_1)}; R_1(b_2); W_1(b_1); \cancel{W_2(b_3)}; \cancel{R_1(b_3)}; \cancel{W_1(b_3)}; R_2(b_4);$
 $W_2(b_4); R_1(b_4); W_1(b_2);$

- T1, b1 bloğu için bekliyor.
- T2, UL(b4) ile bittikten sonra T1 başlayabilir.
- 2PL'ye göre bu planın gerçekleşmesi, **bazı kilitleri önceden alıp, böylece kilit bırakmaya başlayarak olabilir.** (Bir sonraki sunuma bakınız..)

- PLAN2: ?? ($\langle T_1, T_2 \rangle$ seri planına denk gelen bir «serilenebilir» plan yazılamıyor)

- deadlock da olabilir:

$W_1(b_1); R_1(b_2); R_2(b_2); W_1(b_1); R_2(b_3); R_1(b_3);$ deadlock!

$W_1(b_1); R_1(b_2);$

$R_2(b_2);$

$W_1(b_1);$

$R_2(b_3)$

$R_1(b_3);$

$R_2(b_1);$ çalışmıyor

$W_1(b_3);$ çalışmıyor

Örnek –devam (*kilidin önceden alınması*)

$R_2(b_2); R_2(b_3); \underline{R_2(b_1)}; \underline{W_1(b_1)}; R_1(b_2); W_1(b_1); \underline{W_2(b_3)}; \underline{R_1(b_3)}; \underline{W_1(b_3)}; R_2(b_4); W_2(b_4); R_1(b_4); W_1(b_2);$

$SL(b_2); R_2(b_2);$

$XL(b_3); R_2(b_3);$

$SL(b_1); R_2(b_1);$

$XL(b_1);$ bekle

$W_2(b_3);$

$XL(B_4); R_2(b_4); W_2(b_4);$

$U(b_2); U(b_3); U(b_1); U(b_4);$

$XL(b_1); W_1(b_1);$

$XL(b_2); R_1(b_2);$

$W_1(b_1);$

$XL(b_3); R_1(b_3);$

$W_1(b_3);$

$SL(b_4); R_1(b_4);$

$W_1(b_2);$

$U(b_2); U(b_3);$

$U(b_1); U(b_4);$

$XL(b_1); W_1(b_1);$

$XL(b_2); R_1(b_2);$

$W_1(b_1);$

$XL(b_3);$ bekle

$XL(b_3); R_1(b_3);$

$W_1(b_3);$

$SL(b_4);$ bekle

$SL(b_4); R_1(b_4);$

$W_1(b_2);$

$U(b_2); U(b_3);$

$U(b_1); U(b_4);$

$SL(b_2); R_2(b_2);$

$XL(b_3); R_2(b_3);$

$SL(b_1); \underline{R_2(b_1)};$

$XL(B_4); U(b_1); U(b_2);$

$W_2(b_3); U(b_3);$

$R_2(b_4); W_2(b_4);$

$U(b_4);$

Eşzamanlılık Yönetimi

- Serilenebilirlik: (C-S, V-S, DC-S)
- Temel Kilit Protokolü ve 2PL ve versiyonları
- Ölüklilit ve Açlık
- **Kilitler ile Yalıtım seviyelerinin sağlanması**
- Diğer Kilit Protokolleri (Kilit Yükseltme, Update Kilit, Increment kilit, MGL kilit)
- Phantom
- SimpleDB'de eşzamanlılık kontrolü ve Hareket Yönetimi
- Diğer Eşzamanlılık Protokolleri
 - MV-lock
 - TimeStamp
 - Optimistic methods

Kilitler ile Yalıtım seviyelerinin sağlanması

Isolation Level	Problems	Lock Usage	Comments
serializable 2-phase lock	none	slocks held to completion, slock on eof marker	the only level that guarantees correctness
repeatable read	phantoms	slocks held to completion, no slock on eof marker	useful for modify-based transactions
read committed	phantoms, values may change	slocks released early, no slock on eof marker	useful for conceptually separable transactions whose updates are "all or nothing"
read uncommitted	phantoms, values may change, dirty reads	no slocks at all	useful for read-only transactions that tolerate inaccurate results

- Sadece okumada yalıtım seviyeleri tespit edilebilir. "Yazma kilidi", xLock için bir gevşeklik yok: Bütün yalıtım seviyelerinde dışlayıcı kilit hareket sonuna kadar tutulur. (Örneğin: read uncommitted bir hareket slock kullanmadığı için commit olmamaış bir veriyi okuyor. Yoksa dışlayıcı kilit erken bırakılmış bir veriyi okumak değil...)
- read committed'de paylaşımlı anahtar ne zaman serbest bırakılır?
 - Örnek:Oracle hareket içerisinde her bir SQL cümlesi sonunda sLock'u bırakıyor. Böylece her bir SQL içinde bilgi tutarlı fakat farklı SQL sorgulamalarında "repeatable read" korunmuyor.
- Çok versiyon kitleme (MV-lock) ile "read uncommitted" yalıtım seviyesini karşılaştırılması:
 - İkisi de "read" operasyonunda sLock kullanmıyor. Ancak; MV-lock serilenebilir planlar oluştururken, "read uncommitted" hareketler ile serilenemez planlar oluşuyor.
- Update içeren hareketler için "read-uncommitted" niye tehlikeli?"

Eşzamanlılık Yönetimi

- Serilenebilirlik: (C-S, V-S, DC-S)
- Temel Kilit Protokolü ve 2PL ve versiyonları
- Ölüklit ve Açlık
- Kilitler ile Yalıtım seviyelerinin sağlanması
- **Diğer Kilit Protokolleri (Kilit Yükseltme, Update Kilit, Increment kilit, MGL kilit)**
- Phantom
- SimpleDB'de eşzamanlılık kontrolü ve Hareket Yönetimi
- Diğer Eşzamanlılık Protokolleri
 - MV-lock
 - TimeStamp
 - Optimistic methods

Diğer Kilit Protokolleri:

Kilit Yükseltme

- AMAÇ: Gecikmeyi azaltma, Eşzamanlılığı arttırma.
- Hareket çoğu zaman: $R(X); \langle do_s/t_with_X \rangle; W(X)$
 - $XL(X), R(X); \langle do_s/t_with_X \rangle; W(X)$ yerine
 - $SL(X); R(X); \langle do_s/t_with_X \rangle; XL(X), W(X)$

sl1(A); r1(A);

sl2(A); r2(A);
sl2(B); r2(B);

xl1(B) **Bekle**

u2(A); u2(B)

xl1(B); r1(B);
w1(B);
u1(A); u1(B);

sl1(A); r1(A);

sl2(A); r2(A);
sl2(B); r2(B);

sl1(B), r1(B);
xl1(B); **Bekle**

u2(A); u2(B)

xl1(B); w1(B);
u1(A); u1(B);

Kilit Yükseltme → deadlock riski

xl1(A); r1(A);

xl1(A) **Bekle**

sl1(A); r1(A);

sl2(A); r2(A);

W1(A)

xl1(A) **Bekle**

xl1(A) **Bekle**

....

Çözüm önerisi: UPDATE LOCK

«B» Veri parçası için istenen kilit

	S	X	U
«B» Veri parçası üzerinde mevcut kilit			
S	Yes	No	Yes
X	No	No	No
U	No	No	No

- **UL(B);** ile sadece B okunabilir.
- **Sadece UL(B) sonradan XL(B) olabilir.**
- **SL(B) sonradan XL(B) olamaz.**

ul1(A); r1(A);

ul2(A); **Bekle**

xl1(A); w1(A);

U1(A)

ul2(A); r2(A);

xl2(A); w(A); U2(A);

Diğer Kilit Protokolleri: Increment Locks

- D-C Serilenebilirlik sağlamada kullanılabilir.
- **INC(A,c) atomic** olmalı: «READ(A,t); t := t+c; WRITE(A,t);»
- INC(A,c) için gerekli kilit **il(A)**:
 - **il(A); INC(A,c)**
 - ~~il(A); R(A);~~
 - ~~il(A); W(A);~~
 - $inc_i(X)$; hem $r_j(X)$ hem $w_j(X)$ ile gelişiyor.

	S	X	I
S	Yes	No	No
X	No	No	No
I	No	No	Yes

sl1(A); r1(A);

sl2(A); r2(A)

il2(B); inc2(B);

il1(B); inc1(B);

U2(A); U2(B);

U1(A); U1(B)

Çelişki yok!
Bu plan =
<T1,T2> =
<T2,T1>

MGL (multiple granularity locks)

Çok taneşelli kilitleme

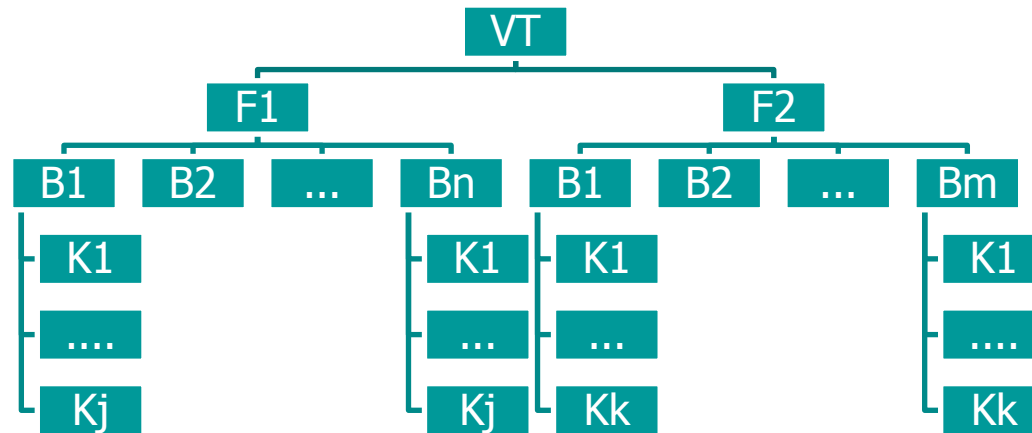
- Tanesellik: Değer < Kayıt < Blok < Tablo < VT



- Tanesellik Değer veya Kayıt ise:
 - h1 ve h2 hareketleri şekildeki gibi eşzamanlı veriyi değiştirebilir. (*çelişki yok*)
 - Kilit sayısı fazla, karmaşıklık fazla
 - UNDO/REDO dışındaki protokollerde diske yazmada sorun olabilir.
- Tanesellik blok seviyesinde ise;
 - çelişki var → eşzamanlılık azaldı
 - Az sayıda kilit, karmaşıklık az
- Az sayıda kayda erişen hareketler (OLTP gibi) için KÜÇÜK TANESELLİK:
- Çok sayıda (veya bütün) kayıtlara erişen hareketler (Veri Ambarı gibi) için BÜYÜK TANESELLİK tercih edilir.
- **Farklı hareket tipleri içiren VT'larında kullanılan kilit protokolü MGL**

MGL (multiple granularity locks)

Çok taneşelli kilitleme



- Senaryo 1:
 - T1, F1 tablosundaki bütün kayıtları değiştirmek istiyor: XL(F1)
 - T2, Kj kaydını okumak istiyor: SL(Kj)
- Senaryo 2:
 - T2, Kj kaydını okumak istiyor: SL(Kj)
 - T1, F1 tablosundaki bütün kayıtları değiştirmek istiyor: XL(F1)
- MGL kilitleri:
 - Normal kilitler: S,X
 - Niyet kilitleri: IS,IX ve (SIX)

MGL

	IS	IX	S	X
IS	Yes	Yes	Yes	No
IX	Yes	Yes	No	No
S	Yes	No	Yes	No
X	No	No	No	No

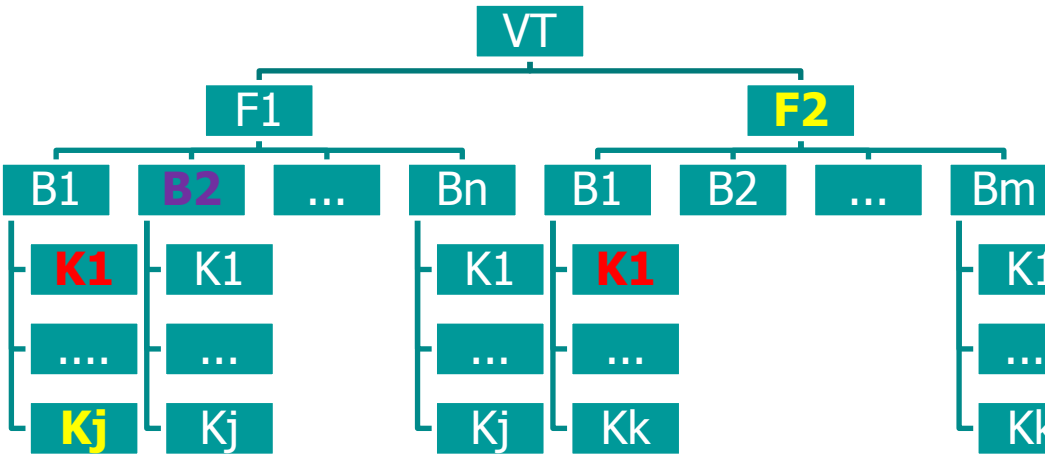
- Herhangi bir seviyede, S veya X kilitlerini alabilmek için her zaman KÖK'ten başlarız.
 - Hedeflenen VT elemanına varmadan evvelki bir «N» VT elemanında, eğer alt elemanlarda
 - **S kilit istenecekse; N için IS kilidi talep edilir.**
 - **X kilit istenecekse; N için IX kilidi talep edilir.**
 - Kilidin hedeflendiği seviyeye varınca, daha ileriye gitmeden kilit bu VT elemanı için alınır.
- Aynı hareket bir VT elemanı için «S» ve «IX» alabilir mi?
 - Bu kilit: SIX. Sadece «IS» ile uyumludur.

Örnek-1

İsim	<u>No</u>	Ders	Not
Ali	10	VT	43
Ahmet	11	VT	32
Osman	12	Derleyiciler	56
Mehmet	13	İşletimSistemi	67

- T1: < **S** * **F** öğrenci **W** ders='VT';>
 - Öğrenci tablosu için **T1, IS kilidini alır.**
 - **10 ve 11 nolu satırlar için «S» kilidini alır.**
- T2: <**update** öğrenci **set** Ders=«compilers» **where** Ders = «Derleyiciler»>
 - Öğrenci tablosu için **T2, IX kilidini alır.**
 - **12 nolu satır için «X» kilidini alır.**

Ornek-2



IX (VT)

IX (F1)

IX (B11)
X (K111)

IX (F2)

IX (B21)
X (K211)

UL (K211)
UL(B21)
UL (F2)

*Diğer
kilitleri
bırakma*

IS (VT)

IS (F1)
IS (B11)

S (K11j)

IX (VT)
IX (F1)
X (B12)

S(F2) **BEKLE**

S(F2)

UL(B12)
UL(F1)
UL(VT)

*Diğer
kilitleri
bırakma*

*Diğer
kilitleri
bırakma*

- T1: K_{111} ve K_{211} kayıtlarını değiştirmek istiyor.
- T2: B_{12} bloğundaki bütün kayıtları değiştirmek istiyor.
- T3: K_{11j} kaydını ve F_2 dosyasını okumak istiyor.

MGL avantajları

- Kısa ve uzun hareekterin çalıştığı ortamlarda **kilit alam/bırakma yükünü** hafifletir.
- **Eşzamanlı çalışmayı** arttırır.

Eşzamanlılık Yönetimi

- Serilenebilirlik: (C-S, V-S, DC-S)
- Temel Kilit Protokolü ve 2PL ve versiyonları
- Ölüklit ve Açlık
- Diğer Kilit Protokolleri (Kilit Yükseltme, Update Kilit, Increment kilit, MGL kilit)
- Kilitler ile Yalıtım seviyelerinin sağlanması
- **Phantom**
- SimpleDB’de eşzamanlılık kontrolü ve Hareket Yönetimi
- Diğer Eşzamanlılık Protokolleri
 - MV-lock
 - TimeStamp
 - Optimistic methods

Phantom problemi

- Daha mevcut olmayan, ileride eklenecek olan bir kayıt önceden kilitleyemediğimiz için bir anda beklenmedik bir şekilde VT'na eklenebilir; bu serilenebilirliğe (*doğruluğa*) zarar verir.
- Örneğin: T1 hareketi «P» yüklemine sağlayan bir kayıt ekliyor. T2 hareketi ise gene «P» yüklemine sağlayan kayıtlar üzerinde bir «kümeleme işlemi» gerçekleştiriyor.
 - $\langle T1, T2 \rangle$ seri planına denk gelen bir planda; T2 yeni kaydı görüp kümeleme fonksiyonuna dahil eder.
 - $\langle T2, T1 \rangle$ seri planına denk gelen bir planda herhangi bir ortak veri yok, çelişki gözüküyor. Çünkü, T2, gerekli kayıtları yeni kayıt gelmeden önce kilitleyip kullanacak. Oysa T1, P yüklemineki şartları sağlayan bir «Phantom kayıt» ekledi. Bu phantom kaydı planın serilenebilirliğine zarar verebilir.

Örnek-1

İsim	No	Ders	Not	X
Ali	10	VT	43	...
Ahmet	11	VT	32	...
Osman	12	Derleyiciler	56	...
Mehmet	13	İşletimSistemi	67	
Hasan	14	VT	90	

T2, VT dersinin not ortalamasını buluyor.

T1, VT dersine ait yeni bir kayıt giriyor.

$r2(K_{10}); r2(K_{11}); w1(K_{14}); r2(K_{14}); w2(AVE);$

Bu plan serilenebilir. $\langle T1, T2 \rangle$

$r2(K_{10}); r2(K_{11}); w1(K_{14}); w2(AVE);$

Bu plan serilenebilir. $\langle T2, T1 \rangle$

$r2(K_{10}); r2(K_{11}); w1(K_{14}); w1(X); w2(AVE); w2(X);$

Bu plan serilenebilir değil. Çünkü $AVE = 43 + 32 / 2$ çünkü $w1(K_{14})$ phantom olduğu için T2 onu göremedi ve okuyamadı. (*planda $R2(K_{14})$ yok!*) O zaman bu noktadan, sadece $\langle T2, T1 \rangle$ planına denk olabilir. Ancak; $w1(X); \dots w2(X);$ de bu denklığı de ortadan kaldırdı. O zaman plan serilenebilir değil.

Örnek-1-çözüm

- Kayıt ekleme işlemi için hareket **bütün dosyada X kilidi** alması gerek. Böylece aynı dosyada okuma yapan diğer hareketler ile çelişki ortaya çıktığı için (önceden gizli olan çelişki) işlemler sıralanabilir.
- Aşağıdaki plan $\langle T3, T4 \rangle$ 'e denk bir plan. (*Bazı kısımları özet geçilmiştir.*) Benzer şekilde $\langle T4, T3 \rangle$ 'e denk planlar yazılabilir.

IS (Öğrenci)

IS (B1)

SL (K_{10}); SL (K_{11})

r3(K_{10}); r3(K_{11});

XL(öğrenci); **BEKLE**

AVE'i hesapla..

XL(X); **w3(X)**;

Kililtleri bırak!

XL(öğrenci); XL(K_{14});

XL (X); w4(X);

Kililtleri bırak!

Örnek-2

- **simpledb'deki (basit) phantom problemi:** T1 append(..) ve T2 size() çelişkisi ile ortaya çıkan bir problem gene bir Phantom roblemidir:
 - **T2 size() fonksiyonunu çok defa üstüste çalıştırır**sa, T1 append(..) işleminin araya girmesi ile, bu okumalardaki değerler farklı olabilir. Bu durum, ACID kurallarına aykırıdır. Serilenebilir (doğru) değil.
 - slock ve xlock tipindeki kilitler sorunu çözmiyor. Çünkü eklenecek kaydı önceden kilitleyemiyoruz.
 - **ÇÖZÜM ÖNERİSİ: Dosya sonu işaretini (EOF marker)** kilitleyebiliriz..
 - Size() fonksiyonu slock (*eof*) yapar.
 - Append(..) fonksiyonu xlock (*eof*) yapar.

Phantom için Diğer çözümler

- **Index Locking:** Veri dosyasında kayıt erişimi ve eklemesi yapan hareketler, erişim indeksinin yaprak düğümünü SL veya XL ile kilitlemesi gerektiği için Phantom kaydı ancak bu noktada yakalanabilir.
- **Predicate Locks (range lock):** SQL cümlesinin yükleminde çelişkileri yakalamak esasına dayanır.
- **Block veya daha büyük tanesellik kullanmak:** Böylece, Phantom yazma mümkün olmayacaktır. Fakat block taneselliği çok iri olduğundan eşzamanlılık az, false conflict çoktur.

Eşzamanlılık Yönetimi

- Serilenebilirlik: (C-S, V-S, DC-S)
- Temel Kilit Protokolü ve 2PL ve versiyonları
- Ölüklit ve Açlık
- Kilitler ile Yalıtım seviyelerinin sağlanması
- Diğer Kilit Protokolleri (Kilit Yükseltme, Update Kilit, Increment kilit, MGL kilit)
- Phantom
- **SimpleDB’de eşzamanlılık kontrolü ve Hareket Yönetimi**
- Diğer Eşzamanlılık Protokolleri
 - MV-lock
 - TimeStamp
 - Optimistic methods

SimpleDB'de EY

- *simpldb.tx.concurrency* paketi
 - Kilit protokolü
 - Blok veri tanesi
- Her bir tx kendisi için 1 adet eşzamanlılık yönetimi kullanır.
- `slock(block)` ve `xlock(block)` metodları tampon tönnetimindeki `pin()` ve `pinNew()` gibi Java `wait()` metodunu kullanır..
- LOCK TABLE için tipik bir API:

```
public void sLock(Block blk);
public void xLock(Block blk);
public void unlock(Block blk);
```
- CONCURRENCY MGR için tipik bir API:

```
public ConcurrencyMgr(int txnum);
public void sLock(Block blk);
public void xLock(Block blk);
public void release();
```

Figure 14-26

The API for the SimpleDB class *ConcurrencyMgr*

LockTable sınıfı

```
class LockTable {
    private static final long MAX_TIME = 10000; // 10 seconds

    private Map<Block,Integer> locks =
        new HashMap<Block,Integer>();

    public synchronized void sLock(Block blk) {
        try {
            long timestamp = System.currentTimeMillis();
            while (hasXlock(blk) && !waitingTooLong(timestamp))
                wait(MAX_TIME);
            if (hasXlock(blk))
                throw new LockAbortException();
            int val = getLockVal(blk);
            locks.put(blk, val+1);
        }
        catch (InterruptedException e) {
            throw new LockAbortException();
        }
    }

    public synchronized void xLock(Block blk) {
        try {
            long timestamp = System.currentTimeMillis();
            while (hasOtherSLocks(blk) &&
                !waitingTooLong(timestamp))
                wait(MAX_TIME);
            if (hasOtherSLocks(blk))
                throw new LockAbortException();
            locks.put(blk, -1);
        }
        catch (InterruptedException e) {
            throw new LockAbortException();
        }
    }

    public synchronized void unlock(Block blk) {
        int val = getLockVal(blk);
        if (val > 1)
            locks.put(blk, val-1);
        else {
            locks.remove(blk);
        }
    }
}
```

Figure 14-27
The code for the SimpleDB class *LockTable*

```
        notifyAll();
    }
}

private boolean hasXlock(Block blk) {
    return getLockVal(blk) < 0;
}

private boolean hasOtherSLocks(Block blk) {
    return getLockVal(blk) > 1;
}

private boolean waitingTooLong(long starttime) {
    long now = System.currentTimeMillis();
    return now - starttime > MAX_TIME;
}

private int getLockVal(Block blk) {
    Integer ival = locks.get(blk);
    return (ival == null) ? 0 : ival.intValue();
}
}
```

Figure 14-27 (Continued)

```

public class ConcurrencyMgr {

    private static LockTable locktbl = new LockTable();
    private Map<Block,String> locks =
        new HashMap<Block,String>();

    public void sLock(Block blk) {
        if (locks.get(blk) == null) {
            locktbl.sLock(blk);
            locks.put(blk, "S");
        }
    }

    public void xLock(Block blk) {
        if (!hasXLock(blk)) {
            sLock(blk);
            locktbl.xLock(blk);
            locks.put(blk, "X");
        }
    }

    public void release() {
        for (Block blk : locks.keySet())
            locktbl.unlock(blk);
        locks.clear();
    }

    private boolean hasXLock(Block blk) {
        String locktype = locks.get(blk);
        return locktype != null && locktype.equals("X");
    }
}

```

Figure 14-28

The code for the SimpleDB class *ConcurrencyMgr*

ConcurrencyManager sinifi

SimpleDB'de HY

- Her hareket (*tx*) için, KY ve EY sınıfı var. LockTable ise ortak.
- Commit ve rollback:
 - Kullanılan tamponları unpin et
 - KY'de commit/rollback
 - EY'de kilitleri serbest bırak
- Her hareket kullandığı tamponları *BufferList* sınıfı ile organize ediyor.

```
public void    commit();
public void    rollback();
public void    recover();

public void    pin(Block blk);
public void    unpin(Block blk);
public int     getInt(Block blk, int offset);
public String  getString(Block blk, int offset);
public void    setInt(Block blk, int offset, int val);
public void    setString(Block blk, int offset, String val);

public int     size(String filename);
public Block   append(String filename, PageFormatter fmtr);
```

Figure 14-2

The API for the SimpleDB class *Transaction*


```

public class Transaction {
    private static int nextTxNum = 0;
    private RecoveryMgr recoveryMgr;
    private ConcurrencyMgr concurMgr;
    private int txnum;
    private BufferList myBuffers = new BufferList();

    public Transaction() {
        txnum = nextTxNumber();
        recoveryMgr = new RecoveryMgr(txnum);
        concurMgr = new ConcurrencyMgr();
    }

    public void commit() {
        myBuffers.unpinAll();
        recoveryMgr.commit();
        concurMgr.release();
        System.out.println("transaction committed");
    }

    public void rollback() {
        myBuffers.unpinAll();
        recoveryMgr.rollback();
        concurMgr.release();
        System.out.println("transaction rolled back");
    }

    public void recover() {
        recoveryMgr.recover();
    }

    public void pin(Block blk) {
        myBuffers.pin(blk);
    }

    public void unpin(Block blk) {
        myBuffers.unpin(blk);
    }

    public int getInt(Block blk, int offset) {
        concurMgr.sLock(blk);
        Buffer buff = myBuffers.getBuffer(blk);
        return buff.getInt(offset);
    }
}

```

Figure 14-29

The code for the SimpleDB class *Transaction*

```

    public String getString(Block blk, int offset) {
        concurMgr.sLock(blk);
        Buffer buff = myBuffers.getBuffer(blk);
        return buff.getString(offset);
    }

    public void setInt(Block blk, int offset, int val) {
        concurMgr.xLock(blk);
        Buffer buff = myBuffers.getBuffer(blk);
        int lsn = recoveryMgr.setInt(buff, offset, val);
        buff.setInt(offset, val, txnum, lsn);
    }

    public void setString(Block blk, int offset,
        String val) {
        concurMgr.xLock(blk);
        Buffer buff = myBuffers.getBuffer(blk);
        int lsn = recoveryMgr.setString(buff, offset, val);
        buff.setString(offset, val, txnum, lsn);
    }

    public int size(String filename) {
        Block dummyblk = new Block(filename, -1);
        concurMgr.sLock(dummyblk);
        return SimpleDB.fileMgr().size(filename);
    }

    public Block append(String filename,
        PageFormatter fmtr) {
        Block dummyblk = new Block(filename, -1);
        concurMgr.xLock(dummyblk);
        Block blk = myBuffers.pinNew(filename, fmtr);
        unpin(blk);
        return blk;
    }

    private static synchronized int nextTxNumber() {
        nextTxNum++;
        System.out.println("new transaction: " + nextTxNum);
        return nextTxNum;
    }
}

```

Figure 14-29 (Continued)


```

class BufferList {
    private Map<Block,Buffer> buffers =
        new HashMap<Block,Buffer>();
    private List<Block> pins = new ArrayList<Block>();
    private BufferMgr bufferMgr = SimpleDB.bufferMgr();

    Buffer getBuffer(Block blk) {
        return buffers.get(blk);
    }

    void pin(Block blk) {
        Buffer buff = bufferMgr.pin(blk);
        buffers.put(blk, buff);
        pins.add(blk);
    }

    Block pinNew(String filename, PageFormatter fmtr) {
        Buffer buff = bufferMgr.pinNew(filename, fmtr);
        Block blk = buff.block();
        buffers.put(blk, buff);
        pins.add(blk);
        return blk;
    }

    void unpin(Block blk) {
        Buffer buff = buffers.get(blk);
        bufferMgr.unpin(buff);
        pins.remove(blk);
        if (!pins.contains(blk))
            buffers.remove(blk);
    }

    void unpinAll() {
        for (Block blk : pins) {
            Buffer buff = buffers.get(blk);
            bufferMgr.unpin(buff);
        }
        buffers.clear();
        pins.clear();
    }
}

```

Figure 14-30

The code for the SimpleDB class *BufferList*

HY Test Örneği

```
public class TransactionTest {
    public static void main(String[] args) {
        //initialize the database system
        SimpleDB.init(args[0]);

        TestA t1 = new TestA(); new Thread(t1).start();
        TestB t2 = new TestB(); new Thread(t2).start();
        TestC t3 = new TestC(); new Thread(t3).start();
    }
}

class TestA implements Runnable {
    public void run() {
        try {
            Transaction tx = new Transaction();
            Block blk1 = new Block("junk", 1);
            Block blk2 = new Block("junk", 2);
            tx.pin(blk1);
            tx.pin(blk2);
            System.out.println("Tx A: read 1 start");
            tx.getInt(blk1, 0);
            System.out.println("Tx A: read 1 end");
            Thread.sleep(1000);
            System.out.println("Tx A: read 2 start");
            tx.getInt(blk2, 0);
            System.out.println("Tx A: read 2 end");
            tx.commit();
        }
        catch (InterruptedException e) {}
    }
}
```

Figure 14-31

Test code for the lock table

```
class TestB implements Runnable {
    public void run() {
        try {
            Transaction tx = new Transaction();
            Block blk1 = new Block("junk", 1);
            Block blk2 = new Block("junk", 2);
            tx.pin(blk1);
            tx.pin(blk2);
            System.out.println("Tx B: write 2 start");
            tx.setInt(blk2, 0, 0);
            System.out.println("Tx B: write 2 end");
            Thread.sleep(1000);
            System.out.println("Tx B: read 1 start");
            tx.getInt(blk1, 0);
            System.out.println("Tx B: read 1 end");
            tx.commit();
        }
        catch (InterruptedException e) {}
    }
}

class TestC implements Runnable {
    public void run() {
        try {
            Transaction tx = new Transaction();
            Block blk1 = new Block("junk", 1);
            Block blk2 = new Block("junk", 2);
            tx.pin(blk1);
            tx.pin(blk2);
            System.out.println("Tx C: write 1 start");
            tx.setInt(blk1, 0, 0);
            System.out.println("Tx C: write 1 end");
            Thread.sleep(1000);
            System.out.println("Tx C: read 2 start");
            tx.getInt(blk2, 0);
            System.out.println("Tx C: read 2 end");
            tx.commit();
        }
        catch (InterruptedException e) {}
    }
}
```

Figure 14-31 (Continued)

Eşzamanlılık Yönetimi

- Serilenebilirlik: (C-S, V-S, DC-S)
- Temel Kilit Protokolü ve 2PL ve versiyonları
- Ölüklit ve Açlık
- Kilitler ile Yalıtım seviyelerinin sağlanması
- Diğer Kilit Protokolleri (Kilit Yükseltme, Update Kilit, Increment kilit, MGL kilit)
- Phantom
- SimpleDB'de eşzamanlılık kontrolü ve Hareket Yönetimi
- **Diğer Eşzamanlılık Protokolleri**
 - **MV-lock**
 - **TimeStamp**
 - **Optimistic methods**

Çok versiyon kilitleme (*multiversion locking*)

- Gerçek hayatta 1 blok üzerine tipik erişim;
 - 1-2 hareket (*okuma-yazma tipinde*)
 - Çok sayıda hareket (*salt-okunur tipte*)
- Çok versiyon kilitleme:
 - Salt-okunur hareketlerde, okuma için kilit (*slock*) kullanılmıyor. Okunmak istenen bloğun, (*read uncommitted durumuna düşmeyeceği*) **tutarlı bir versiyonunun** okunması gerekir. Bu arada bu blok üzerinde diğer hareketler eşzamanlı değişiklik yapabiliyor.
 - **tutarlı bir versiyon:** hareketin başlamasından önceki en son tutarlı (commit edilmiş) blok versiyonu.
- Bir hareket *onaylanıp(commit)* sonlandığı zaman, değişiklik yaptığı bloklar blok versiyonu oluşturur. Her blok versiyonu, versiyonun olduğu zaman bilgisi (*versiyonu oluşturan hareketin commit olma zamanı*) ve hangi hareketin sonlanarak bu versiyonu oluşturduğu bilgisini saklar.
- Hareketin "readOnly" olması Connection kurulurken belirtilmeli; ta ki MV eşzamanlılığına imkan sağlayabilelim: **conn.setReadOnly(true);**

MV-lock
ÖRNEK:
plan, hangi
seri çalışmaya
karşılık
geliyor?

Consider four transactions having the following histories:

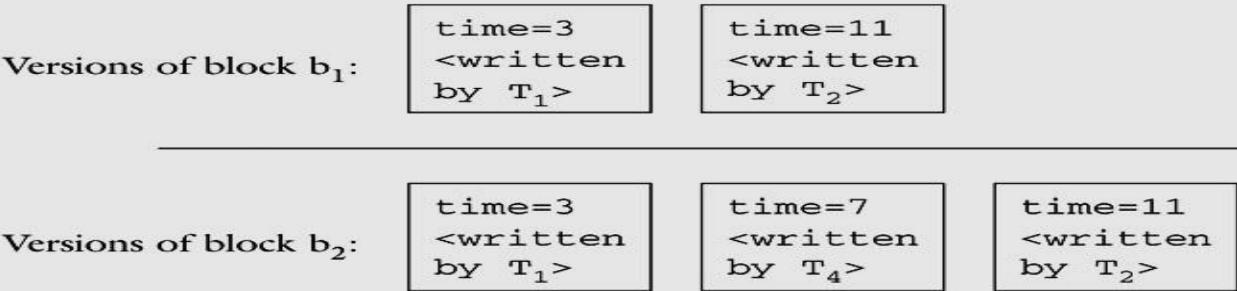
$T_1: W(b_1); W(b_2);$
 $T_2: W(b_1); W(b_2);$
 $T_3: R(b_1); R(b_2);$
 $T_4: W(b_2);$

Suppose that these transactions execute according to the following schedule:

$W_1(b_1); W_1(b_2); C_1; W_2(b_1);$ $R_3(b_1);$ $W_4(b_2); C_4;$
 $R_3(b_2);$ $C_3; W_2(b_1); C_2;$

In this schedule we assume that a transaction begins where its first operation is, and use C_i to indicate when transaction T_i commits. The update transactions T_1 , T_2 , and T_4 follow the lock protocol, as you can verify from the schedule. Transaction T_3 is a read-only transaction and does not follow the protocol.

The concurrency manager stores a version of a block for each update transaction that writes to it. Thus there will be two versions of b_1 and three versions of b_2 :



The time stamp on each version is the time when its transaction commits, not when the writing occurred. We assume that each operation takes one time unit, so T_1 commits at time 3, T_4 at time 7, T_3 at time 9, and T_2 at time 11.

Now consider the read-only transaction T_3 . It begins at time 5, which means that it should see the values that were committed at that point—namely, the changes made by T_1 but not T_2 or T_4 . Thus, it will see the versions of b_1 and b_2 that were time-stamped at time 3. Note that T_3 will not see the version of b_2 that was time-stamped at time 7, even though that version had been committed by the time that the read occurred.

Figure 14-24
An example of multiversion locking

Zaman damgalı eşzamanlılık kontrolü (*Timestamp concurrency control*)

- $TS(T)$: hareket başlama zamanı
- P planındaki hareketlerin $TS(T_i)$ zamanları farklı.
- P planında $TS(T_i) < TS(T_j) < TS(T_k)$ ise;
 - denk geldiği «seri plan»: $\langle T_i .. T_j .. T_k \rangle$ dir.
 - Çelişen operasyonlar da(R-W, W-W), veri parçaları üzerinde bu sırada işlem yapmalıdırlar.: Bunu sağlamak için readTS(X) ve write TS(X) tanımlanır.
- readTS(X): X verisinde okuma yapan en genç (TS değeri en büyük olan) hareketin TS değeri.
- write TS(X): X verisinde yazma yapan en genç (TS değeri en büyük olan) hareketin TS değeri.

Temel Zaman damga protokolü

- T, write_item(X):
 - $\text{read_TS}(X) > \text{TS}(T)$ veya $\text{write_TS}(X) > \text{TS}(T)$ ise işlemi geri çevir; ve abort/rollback T.
 - Birinci adımdan geçtiyse; işlemi çalıştır ve **$\text{write_TS}(X) = \text{TS}(T)$**
- T, read_item(X):
 - $\text{write_TS}(X) > \text{TS}(T)$ işlemi geri çevir; ve abort/rollback T.
 - $\text{write_TS}(X) \leq \text{TS}(T)$ ise çalıştır ve **$\text{read_TS}(X) = \max(\text{TS}(T), \text{read_TS}(X))$**
- **Thomas's Write Rule:** T, write_item(X) işlemini geri çevirmede hafifleştirme:
 - $\text{read_TS}(X) > \text{TS}(T)$ ise geri çevir.
 - $\text{write_TS}(X) > \text{TS}(T)$ ise write_item(X)'i işlemeden atla. Çünkü daha genç bir hareketin yaptığı yazma bunu geçersiz kıldı.

Örnek

- Örnek eklenmesi planlanıyor..

«TS concurrency» Dezavantajları

- Ti abort/rollback olursa ve Tj, Ti'den okuduysa Tj de rollback olmalı.
 - Eğer Ti'den okuyan Tj, Ti'den önce commit olduysa daha kötü → kurtarılamaz plan
- PROBLEM: Zincirli rollback (cascade rollback)
- ÇÖZÜM:
 - Hareketin kendi iç planında yazmaları sona ertelemek
 - All writes of a transaction form an atomic action; no transaction may execute while a transaction is being written ???
 - A transaction that aborts is restarted with a new timestamp ???